molena.huynh@jmp.com

Uncertainty-Calibrated Trust Regions for Query-Efficient QAOA Parameter Search

Molena Huynh

North Carolina State University, Raleigh, NC 27695, USA

June 11, 2026

Abstract

For low-depth implementations of the Quantum Approximate Optimization Algorithm (QAOA), the number of objective-function queries required to identify useful variational parameters is a dominant practical cost. We present UQ-QAOA, a framework that converts graph-conditioned predictive uncertainty into an operational trust-region geometry for query-efficient parameter search. A Graph Isomorphism Network predicts a diagonal Gaussian over QAOA angles; four information sources—neural predictor, local k-nearest-neighbour prior, global population prior, and a trotterized quantum annealing schedule—are fused via precision-weighted combination to obtain a posterior mean and anisotropic step sizes for sequential coordinate refinement. On an exact-statevector MaxCut benchmark (four graph families, n=14, p=3, Q=18 matched queries), UQ-QAOA achieves a mean approximation ratio of 0.865 versus 0.853 for the strongest single-source baseline (paired advantage +0.012, bootstrap 95 % CI $[+0.003, +0.024]$). The method is best-or-tied-best at every intermediate query budget. All results are reproducible from deterministic scripts and a public code artifact.

1 Introduction

The Quantum Approximate Optimization Algorithm (QAOA) is a variational quantum algorithm for combinatorial optimization in which a parameterized quantum circuit is optimized by a classical outer loop[1–3]. For near-term implementations, the dominant computational cost is often the number of objective-function queries required to tune the variational parameters[4–8]. A growing body of work treats QAOA performance evaluation, parameter transfer, hardware execution, circuit cutting, and algorithm selection as resource-accounting problems rather than only final-objective comparisons[9–13]. Related studies emphasize low-depth ansatz design, expectation-value landscapes, parameter clustering, and sampling efficiency[14–17]. In that same spirit, this manuscript poses a bounded, resource-aware question at the interface of machine learning and quantum optimization: given a fixed number of objective queries, can graph-conditioned uncertainty allocate those queries more effectively than single-source warm starts or generic black-box optimizers? Recent studies have demonstrated QAOA performance gains through parameter strategies[18,19], reinforcing the importance of query-efficient classical outer loops. Reducing the number of objective evaluations is therefore a central algorithmic objective for low-depth QAOA, especially given the observation that barren plateaus and noise-induced effects can render gradient-based strategies inefficient[20,21].

The problem addressed here is not only parameter prediction; it is measurement and query allocation in variational quantum algorithms. In near-term settings, the query count—the number of distinct angle vectors evaluated by the quantum processor or simulator—is a first-class resource that constrains the practical utility of QAOA as quantum software[6,7]. A method that transforms learned uncertainty into operational search geometry is therefore a contribution at the machine-learning/quantum-optimization interface: it provides a principled mechanism for trading off exploration width against a finite evaluation budget.

The key novelty is not "a GNN predicts QAOA angles"—that capability exists[22,23]—but rather that the *graph-conditioned predictive uncertainty* is repurposed as an *operational trust-region geometry* for query-efficient parameter search. Concretely, the predicted diagonal covariance determines (i) the anisotropic step sizes of greedy coordinate refinement, (ii) the admissible search ellipsoid, and (iii) instance-adaptive budget allocation, thereby connecting learned structure to query-policy decisions through an explicit geometric mechanism.

This work introduces UQ-QAOA, a graph-conditioned trust-region framework for query-efficient QAOA. Given a graph instance G, a graph neural network predicts a Gaussian distribution

$$q_\phi(\theta \mid G) = \mathcal{N}(\mu_\phi(G), \Sigma_\phi(G)) \tag{1}$$

over the QAOA parameters $\theta \in \mathbb{R}^{2p}$. The predicted mean $\mu_\phi(G)$ initializes local optimization, while the covariance $\Sigma_\phi(G)$ defines an ellipsoidal trust region that constrains the subsequent search and helps allocate an instance-dependent evaluation budget. The learned model thus defines a search policy rather than only a point estimate.

The approach is distinct from several related but different strategies: *parameter-transfer heuristics*[5,24] predict a single parameter vector without search geometry; *GNN warm-start methods*[22,23] use neural predictions as initializations without covariance-guided refinement; *black-box optimizers*[8,25] build per-instance surrogates from scratch at each test instance without cross-instance amortization; *classical trust-region methods*[26,27] derive their region from local second-order information (Hessian or model gradient) rather than from cross-instance learned structure; and *Bayesian optimization*[28,29] constructs a per-instance GP surrogate from test-time evaluations, whereas our method amortizes the trust region across instances via a trained GNN and spends the query budget on direct objective evaluation rather than surrogate refinement.

The motivation comes from the observed structure of QAOA parameter landscapes: for fixed depth and bounded-degree graph families, QAOA parameters exhibit concentration and transferability across related instances[10,30–34]. A learned trust-region policy exploits this transferable structure while avoiding unnecessary exploration. The graph-conditioned component uses a Graph Isomorphism Network (GIN)[35,36] augmented with spectral positional information[37–39].

The contributions of this paper are as follows.

1. **Uncertainty as search geometry.** We use graph-conditioned covariance not as a confidence score alone, but as the metric that defines anisotropic trust-region scales and the step sizes of coordinate refinement in QAOA angle space. This converts a

learned distributional object into three operational decisions: where to center, how wide to search per coordinate, and how to distribute a fixed query budget.

2. **Posterior fusion of learned and physics-informed priors.** The search policy combines a GIN predictor, local k-NN information, a global population prior, and a TQA prior into a single posterior mean and diagonal covariance via precision-weighted (inverse-variance) combination. The fusion formula is explicit and non-iterative.

3. **Controlled matched- and dimension-scaled-budget evidence.** The main benchmark uses depth p=3 with matched query curves, while p=4–7 experiments use dimension-scaled budgets $Q_p = 5 + 4p$ and additional query-curve points for validation. All evaluation is exact-statevector on unweighted MaxCut instances; claims are bounded to this CPU-only evidence.

4. **Fair local-search baselines and ablations.** We include TQA with the identical greedy coordinate-refinement protocol, along with ablations that remove covariance, shuffle covariance, remove individual priors, or disable refinement. We also include standard black-box optimizers (DE, CMA-ES, GP-EI, Nelder–Mead) under the same budget.

5. **Calibration and finite-shot diagnostics.** Because uncertainty is the central object, we report trust-region coverage, calibration diagnostics, and finite-shot/noisy-objective sensitivity where available.

6. **Auditable computational artifact.** The scientific results are generated by deterministic Python scripts (`code/generate_all.py`), while a C++20 reference interface and benchmark record the evaluator contract, compiler settings, thread-level scaling behavior, and kernel structure required for future optimized threaded or accelerator-backed implementations. The code separates candidate generation from evaluator kernels to support future hardware or accelerated backends without changing the algorithmic comparison.

Manuscript organization. Section 2 defines the MaxCut/QAOA objective and the query-budget model. Section 3 places the work relative to QAOA transfer, learning-assisted initialization, Bayesian optimization, trust-region methods, uncertainty quantification, and closely related learning-assisted QAOA work. Sections 4–6 describe the posterior-fusion policy and the limited guarantees used to interpret it. Sections 7–8 give the research design and bounded exact-statevector evidence. Sections 9 and 10 summarize limitations, reproducibility, and future work.

2 Problem Setting and Preliminaries

2.1 MaxCut and the QAOA objective

Let $G = (V, E)$ be an undirected graph with $n = |V|$ vertices. The MaxCut cost Hamiltonian is

$$H_C(G) = \sum_{(i,j)\in E} \frac{1 - Z_i Z_j}{2}. \tag{2}$$

At QAOA depth p, the parameter vector is $\theta = (\gamma_1, \ldots, \gamma_p, \beta_1, \ldots, \beta_p) \in \Theta_p$, where the parameter domain is

$$\Theta_p = [0, 2\pi)^p \times [0, \pi)^p \subset \mathbb{R}^{2p}, \tag{3}$$

corresponding to the 2π-periodicity of the cost phase and π-periodicity of the mixer rotation. In the implementation, angles that leave these domains are projected back by modular wrapping. The QAOA state is

$$|\psi_G(\theta)\rangle = \prod_{\ell=1}^{p} e^{-i\beta_\ell H_M} e^{-i\gamma_\ell H_C(G)} |+\rangle^{\otimes n}, \tag{4}$$

with mixer $H_M = \sum_{i=1}^{n} X_i$, and the exact QAOA objective is

$$f_G(\theta) = \langle \psi_G(\theta) | H_C(G) | \psi_G(\theta) \rangle. \tag{5}$$

The approximation ratio is $r_G(\theta) = f_G(\theta)/C_G^\star$, where $C_G^\star$ is the optimal cut value.

2.2 Objective-query model

We model QAOA optimization as an interaction between a classical optimizer and an objective oracle. One exact query returns $f_G(\theta)$; in the finite-shot model, a noisy estimate is

$$\widehat{f}_{G,S}(\theta) = \frac{1}{S} \sum_{a=1}^{S} C_G(z_a), \quad z_a \sim |\langle z_a | \psi_G(\theta) \rangle|^2. \tag{6}$$

The cost of an optimization run is measured by the number of objective queries $Q = \#\{\theta \text{ evaluated}\}$ and, in the finite-shot setting, by the total shot budget $B = \sum_{q=1}^{Q} S_q$. Algorithms are evaluated through the query-performance curve $Q \mapsto \max_{1 \le q \le Q} r_G(\theta_q)$.

A method that achieves a slightly higher objective only after many more queries may be less useful in a measurement-limited regime than one that reaches comparable quality using substantially fewer queries. We therefore evaluate algorithms both by their best approximation ratio at a fixed budget and by the query count required to reach a target ratio τ:

$$Q_G(\tau) = \min\{q : r_G^{\text{best}}(q) \ge \tau\}. \tag{7}$$

2.3 Learning-assisted parameter selection

A common strategy is to train a point predictor $h_\phi : G \mapsto \widehat{\theta}_G$. Such a predictor may provide a useful warm start, but does not answer two operational questions: how far from the prediction should the optimizer search, and how many queries should be spent on that instance?

To address these, we consider probabilistic predictors $h_\phi : G \mapsto (\mu_\phi(G), \Sigma_\phi(G))$, where the induced distribution is $\theta \mid G \sim \mathcal{N}(\mu_\phi(G), \Sigma_\phi(G))$. The predicted mean is a warm start; the covariance constructs the local search region.

2.4 Graph-conditioned trust regions

A probabilistic predictor maps a graph to a Gaussian: $h_\phi : G \mapsto (\mu_\phi(G), \Sigma_\phi(G))$. The induced trust region is

$$\mathcal{T}_{\phi,\rho}(G) = \left\{ \theta \in \Theta_p : (\theta - \mu_\phi(G))^\mathsf{T} (\Sigma_\phi(G) + \lambda I)^{-1} (\theta - \mu_\phi(G)) \le \rho^2 \right\}, \tag{8}$$

where $\rho > 0$ is a radius parameter and $\lambda > 0$ ensures numerical stability. For diagonal covariance, each QAOA angle is searched in an interval scaled by its predicted uncertainty. The local search problem is $\max_{\theta \in \mathcal{T}_{\phi,\rho}(G)} f_G(\theta)$.

2.5 Trotterized quantum annealing schedule

The trotterized quantum annealing (TQA) schedule provides a physics-informed parameter family that requires no test-instance evaluation. It discretizes a linear adiabatic interpolation between the mixer and cost Hamiltonians into p Trotter steps of size $\Delta t = T/p$[4,33]. For total annealing time T and step index $\ell \in \{1, \ldots, p\}$, the schedule angles are

$$\gamma_\ell^{\mathrm{TQA}} = \frac{\ell}{p} \Delta t, \qquad \beta_\ell^{\mathrm{TQA}} = \left(1 - \frac{\ell}{p}\right) \Delta t, \tag{9}$$

so that the cost-phase weight ramps up linearly while the mixer weight ramps down. The single scalar T is selected on validation data and held fixed across test instances. Because (9) depends only on the depth p and not on the specific test graph, every method in the benchmark has equal access to the TQA candidate; this is the property that makes the TQA safety prefix of UQ-QAOA a fair early-budget control (section 4).

2.6 Graph feature vector

The local k-nearest-neighbour prior and the heuristic baseline operate on a fixed-dimensional descriptor $f(G) \in \mathbb{R}^8$ summarizing graph structure. Each component is invariant to vertex relabelling and is normalized to a comparable scale across graph sizes:

$$f(G) = \big(n,\ m/\tbinom{n}{2},\ \bar{d}/(n-1),\ \sigma_d,\ C,\ \lambda_2,\ \lambda_{\max},\ \kappa\big), \tag{10}$$

where $n = |V|$, $m = |E|$, $\bar{d}$ and σ_d are the mean and standard deviation of the degree sequence, C is the global clustering coefficient, λ_2 and $\lambda_{\max}$ are the algebraic connectivity and the largest eigenvalue of the normalized Laplacian, and κ is the degree assortativity. Nearest neighbours in algorithm 1 are computed by Euclidean distance in this standardized feature space, with per-coordinate z-scoring estimated on the training library.

3 Relation to Prior Work

Parameter concentration and transfer. QAOA parameters exhibit concentration and transferability across related instances for fixed depth and bounded-degree families[1,3,4,10,16,24,30,31]. This has motivated transfer heuristics based on interpolation, nearest-neighbor matching, and parameter databases[5,24,33,40–42]. Warm-starting from classical solutions[43] and building spatial symmetries into the circuit[44,45] are complementary approaches for reducing search cost. Our work builds on this premise but addresses a different question: rather than producing a single transferred parameter vector, we learn a graph-conditioned distribution whose covariance determines the search geometry. The posterior / prior fusion explicitly incorporates transferability information (via the k-NN and global priors) together with a physics-informed TQA schedule, but converts these into a joint covariance that governs step sizes—not merely a single point estimate.

Learning-based initialization. Neural networks and graph neural networks have been used to predict initial QAOA angles from instance data[22,23,46,47]. These methods typically evaluate success through point-prediction quality (e.g., cosine similarity or mean-squared error between predicted and optimal parameters). More broadly, learning to optimize combinatorial problems with GNNs has received significant attention[48–52], and amortized optimization[53–55] provides a general framework for training models that map problem instances to solutions. Meta-learning[56] offers an alternative paradigm for rapid adaptation. Our method changes the role of the learned model: it returns not only a warm-start point but an uncertainty estimate $\Sigma_\phi(G)$ that constructs the local optimization domain and determines direction-dependent search widths, connecting initialization to query allocation.

Bayesian and derivative-free optimization. Bayesian optimization, evolutionary methods, and derivative-free optimizers have been studied as classical outer loops[8,25,28,29]. The Nelder–Mead simplex method[57] remains a common derivative-free choice despite its known limitations in moderate dimension. Standard BO builds a per-instance GP surrogate from objective evaluations on the current instance; each test-time query refines the surrogate before selecting the next point. Our method uses a graph-conditioned model trained offline to propose a trust region *before* any test-time queries, providing an amortized cross-instance prior that requires no per-instance surrogate fitting. The query budget is spent directly on objective evaluation rather than on surrogate refinement. This is complementary: the learned trust region can serve as an informed initialization for subsequent Bayesian optimization if additional budget becomes available.

Trust-region methods. Classical trust-region methods[26,27] restrict local steps to regions where a model is reliable. Our approach differs in two ways: (i) the region is predicted from the graph (not from local Taylor approximation or a Hessian estimate computed during optimization), and (ii) the geometry is probabilistic (anisotropic covariance determines direction-dependent search widths rather than a uniform radius). Like classical trust-region contraction, our method halves step sizes upon stagnation; unlike it, the initial geometry is learned cross-instance rather than inferred from local curvature.

Uncertainty quantification in deep learning. Predictive uncertainty has been modeled through dropout[58], deep ensembles[59], heteroscedastic regression[60,61], and calibrated regression[62]. Surveys by Abdar et al.[63], Hüllermeier and Waegeman[64], and Psaros et al.[65] provide comprehensive overviews. Our work instantiates heteroscedastic regression for QAOA angles and uses the predicted variance as operational search geometry.

Conformal prediction. Distribution-free coverage guarantees through conformal prediction[66–68] have motivated our conformal trust-region radius (Proposition 1).

Measurement cost and resource accounting. A growing body of work emphasizes that variational quantum algorithms should be assessed by the quantum resources required to obtain a given value[6–9,13,19,69]. We adopt this perspective: performance is reported as a function of objective-query count under matched budgets.

Closely related work in learning-assisted and resource-aware QAOA. Several studies provide the closest methodological context for the present work. Larkin et al. evaluate QAOA for MaxCut using approximation-ratio and sampling-based metrics, emphasizing that practical performance depends on the cost of obtaining useful samples rather than on an idealized final objective alone[9]. Streif and Leib show that QAOA parameters can be inferred classically and transferred, motivating outer-loop reduction through problem-structure information[10]. Moussa, Calandra, and Dunjko frame near-term quantum optimization as an algorithm-selection problem in which machine learning identifies instances where QAOA is expected to be useful[11]. Vijendran et al. and Ozaeta et al. report bounded low-depth QAOA claims supported by explicit MaxCut/Ising benchmarks and landscape analysis[14,15]. Bechtold et al. provide a useful model for research-question framing, resource accounting, limitations, and data-availability reporting in QAOA experiments on NISQ devices[13]. We follow the same conservative pattern: state the resource question, control the benchmark, expose the data/code path, and keep hardware and advantage claims outside the evidence.

GNN architectures for structured prediction. Message-passing neural networks[70], GraphSAGE[71], and structure-aware variants[72,73] have established graph neural networks as expressive function approximators over combinatorial structures. Our choice of GIN[35] is motivated by its maximally expressive injective aggregation among first-order message-passing schemes.

High-performance statevector simulation and artifact reproducibility. Large-scale statevector simulation is now standard infrastructure for variational quantum algorithm research[74–76]. At the artifact level, reproducibility requires explicit numerical contracts: deterministic candidate ordering, fixed floating-point reduction semantics, and bitwise-reproducible evaluator output under identical seeds. We use a small-n reference evaluator ($n \leq 18$) deliberate for algorithmic auditing; the kernel boundary documented in section 5 enables future backends—CPU threaded, GPU, or task-runtime—to replace the evaluator without changing the scientific comparison.

Gap addressed by this work. Existing learning-assisted QAOA works predict parameters or provide warm starts but do not use the predicted uncertainty as a test-time resource-allocation object. The central novelty of UQ-QAOA is that the learned covariance constructs the local search geometry and the dimension-wise step sizes for refinement, connecting initialization quality to query-budget distribution through an explicit geometric mechanism. Specifically: (i) the TQA schedule is used both as one of four prior sources *and* as an early-budget safety prefix, separating the contribution of posterior fusion from the strongest physics-informed anchor; (ii) no full GP surrogate is maintained—the method is not Bayesian optimization in disguise, but a learned-geometry local-search policy; (iii) the method does not require gradient evaluations or Hessian estimates from the quantum backend.

4 Graph-Conditioned Gaussian Prediction and Trust-Region Search

4.1 Overview

For each graph G, the algorithm constructs $q_\phi(\theta \mid G) = \mathcal{N}(\mu_\phi(G), \Sigma_\phi(G))$ and defines a trust region $\mathcal{T}_{\phi,\rho}(G)$. At test time, a small number of objective evaluations inside $\mathcal{T}_{\phi,\rho}(G)$ are performed and the best parameter is returned.

4.2 Graph representation

We represent $G = (V, E)$ by its adjacency matrix and node features (normalized degree, clustering statistics). A Graph Isomorphism Network (GIN)[35,36,70] with learnable ε and two-layer MLPs with layer normalization[77] performs message passing:

$$h_i^{(\ell+1)} = \mathrm{MLP}^{(\ell)}\left((1 + \varepsilon^{(\ell)})\, h_i^{(\ell)} + \sum_{j \in \mathcal{N}(i)} \tilde{A}_{ij}\, h_j^{(\ell)} \right), \tag{11}$$

where $\tilde{A} = D^{-1}(A + I)$. A graph-level embedding is obtained by concatenating mean and max pooling of the final node embeddings:

$$g_\phi(G) = \left[\tfrac{1}{n} \textstyle\sum_i h_i^{(L)} \; ; \; \max_i h_i^{(L)} \right]. \tag{12}$$

4.3 Gaussian prediction head

The graph embedding is mapped to a mean $\mu_\phi(G) = W_\mu\, g_\phi(G) + b_\mu$ and diagonal covariance $\Sigma_\phi(G) = \mathrm{diag}(\sigma^2_{\min} + \mathrm{softplus}(a_{\phi,k}(G)))$.

4.4 Training objective

The primary training loss is the Gaussian negative log-likelihood:

$$\mathcal{L}_{\mathrm{nll}}(\phi) = \frac{1}{2N} \sum_{i=1}^{N} \sum_{k=1}^{2p} \left[\frac{(\theta^{\mathrm{ref}}_{i,k} - \mu_{\phi,k}(G_i))^2}{\sigma^2_{\phi,k}(G_i)} + \log \sigma^2_{\phi,k}(G_i) \right], \tag{13}$$

with weight-decay regularization: $\mathcal{L}(\phi) = \mathcal{L}_{\mathrm{nll}}(\phi) + \eta_{\mathrm{wd}} \|\phi\|_2^2$. Reference parameters θ_i^{ref} are obtained by offline high-budget optimization on training graphs.

The first term encourages accurate prediction; the second penalizes unnecessary covariance inflation. A model that underestimates uncertainty incurs a large penalty when reference parameters fall outside the predicted region; a model that inflates uncertainty everywhere produces trust regions that are too large to be query-efficient.

Remark 1. *In the submitted artifact, the GIN is trained with Gaussian NLL and weight decay only. At the tested scale (*240 *training graphs, p=*3*), this produces sufficiently calibrated covariances for the four-source posterior. An explicit calibration regularizer is a principled extension for settings where NLL alone underestimates uncertainty. Graph contrastive learning*[73,78–81] *provides a potential augmentation strategy for learning richer graph embeddings in the data-limited regime.*

4.5 Adaptive query budget

The covariance determines how many queries are spent on each instance. Define a scalar uncertainty score $u_\phi(G) = \frac{1}{d}\mathrm{tr}\,\Sigma_\phi(G)$ and allocate:

$$Q_G = Q_{\min} + \left\lceil (Q_{\max} - Q_{\min}) \frac{u_\phi(G) - u_{\min}}{u_{\max} - u_{\min} + \varepsilon} \right\rceil. \tag{14}$$

This allocates more evaluations to uncertain instances. For fair comparison, adaptive methods are evaluated under matched average budget: $\sum_G Q_G = |\mathcal{D}_{\text{test}}|\, Q_{\text{avg}}$.

4.6 Trust-region construction

Definition 1 (Trust region). $\mathcal{T}_{\phi,\rho}(G) = \left\{\theta \in \Theta_p : (\theta - \mu_\phi(G))^\mathsf{T}(\Sigma_\phi(G) + \lambda I)^{-1}(\theta - \mu_\phi(G)) \leq \rho^2\right\}$.

The radius ρ is selected on a validation set. For diagonal covariance:

$$\sum_{k=1}^{2p} \frac{(\theta_k - \mu_{\phi,k}(G))^2}{\sigma^2_{\phi,k}(G) + \lambda} \leq \rho^2. \tag{15}$$

4.7 Four-source posterior and algorithm

The trust region fuses four independent sources via precision-weighted combination. The GIN predicts $(\mu_{\text{GIN}}, \Sigma_{\text{GIN}})$. The local prior uses the median and variance of optimal angles from the k=5 nearest training graphs. The global prior uses training-library statistics. The TQA prior centers on the linear-ramp schedule. The posterior is:

$$\Sigma_{\text{post}}^{-1} = {\Sigma'_{\text{GIN}}}^{-1} + \Sigma_{\text{loc}}^{-1} + \Sigma_{\text{glob}}^{-1} + \Sigma_{\text{TQA}}^{-1}, \tag{16}$$

where Σ'_{GIN} is calibrated by flooring at the local prior variance. Using TQA as a prior source is equitable: the TQA schedule is computed from depth p without consulting test-instance evaluations, so every method has access to the same schedule information. The budget is spent on objective evaluations in Phase 2.

The algorithm proceeds in three budget-accounted stages. Phase 0 evaluates a short TQA safety prefix. This makes the proposed policy dominance-preserving with respect to the strongest physics-informed early-budget baseline: at each early query count, UQ-QAOA has evaluated the same TQA candidate available to the TQA curve before spending additional queries on learned geometry. Phase 1 then evaluates nonduplicate deterministic posterior anchors (TQA, posterior mean, GIN, local k-NN median, global median). Phase 2 performs sequential greedy coordinate refinement around the empirically best anchor: for each dimension j, the algorithm probes $\pm\delta_j$ from the current best, updating the incumbent on improvement. After a full pass without improvement, step sizes are halved (trust-region contraction). Step sizes are $\delta_j = \mathrm{clip}(0.5\sqrt{[\Sigma_{\text{post}}]_{jj}},\, 0.05,\, 0.30)$.

The key algorithmic distinction is that UQ-QAOA first preserves the strongest physics-informed early-budget behavior through the TQA safety prefix, then *sequentially adapts* remaining probes toward the most promising learned trust-region anchor—unlike single-source baselines, which pre-commit all candidates without posterior fusion.

Algorithm 1 UQ-QAOA: Phase 1 — four-source posterior construction

Require: Trained predictor h_ϕ; training library $\mathcal{L}$; graph G with features $f(G)$; depth p; budget Q_G

Ensure: Posterior mean μ_{post}, step sizes δ_j, anchors

— **Source 1: GIN prediction** —

$(\mu_{\text{GIN}}, \Sigma_{\text{GIN}}) \leftarrow h_\phi(G)$

— **Source 2: Local k-NN prior** —

$\mathcal{N}_k \leftarrow k$-nearest neighbors of $f(G)$ in $\mathcal{L}$

$\mu_{\text{loc}} \leftarrow \text{median}\{\theta_i^* : i \in \mathcal{N}_k\}$; $\Sigma_{\text{loc}} \leftarrow \text{diag}(\text{var}\{\theta_i^* : i \in \mathcal{N}_k\})$

— **Source 3: Global population prior** —

$\mu_{\text{glob}} \leftarrow \text{median}\{\theta_i^* : i \in \mathcal{L}\}$; $\Sigma_{\text{glob}} \leftarrow \text{diag}(\text{var}\{\theta_i^* : i \in \mathcal{L}\})$

— **Source 4: TQA physics-informed prior** —

$\theta_{\text{TQA}} \leftarrow$ TQA linear ramp at depth p

$\Sigma_{\text{TQA}} \leftarrow \text{diag}(\text{med}_{i\in\mathcal{L}}(\theta_i^* - \theta_{\text{TQA}})^2)$

— **Posterior fusion** —

$\Sigma'_{\text{GIN}} \leftarrow \max(\Sigma_{\text{GIN}},\ \Sigma_{\text{loc}})$

$\Sigma_{\text{post}}^{-1} \leftarrow {\Sigma'_{\text{GIN}}}^{-1} + \Sigma_{\text{loc}}^{-1} + \Sigma_{\text{glob}}^{-1} + \Sigma_{\text{TQA}}^{-1}$

$\mu_{\text{post}} \leftarrow \Sigma_{\text{post}}({\Sigma'_{\text{GIN}}}^{-1}\mu_{\text{GIN}} + \Sigma_{\text{loc}}^{-1}\mu_{\text{loc}} + \Sigma_{\text{glob}}^{-1}\mu_{\text{glob}} + \Sigma_{\text{TQA}}^{-1}\theta_{\text{TQA}})$

$\delta_j \leftarrow \text{clip}(0.5\sqrt{[\Sigma_{\text{post}}]_{jj}},\ 0.05,\ 0.30)$

Precision-weighted fusion and independence approximation. The four-source combination is a "precision-weighted posterior-style" fusion using the inverse-covariance weighting formula from independent Gaussian evidence. The sources are not truly independent Bayesian likelihoods; the k-NN and GIN sources share training data, and the TQA schedule is available to all methods. We adopt this engineering approximation because it produces a well-defined center and step-size vector with a clear geometric interpretation: sources with smaller predicted variance exert stronger pull on the joint estimate. In the ablation (table 5), the fusion outperforms any single source, confirming that the approximation is operationally effective.

Why the k-NN prior dominates some ablations. Removing the k-NN prior produces the largest isolated performance drop ($0.865 \to 0.644$, table 5). This occurs because the k-NN median provides a strong anchor in structured graph families: when the test graph has nearby training neighbors, their optimized parameters are already near-optimal. The GIN alone cannot reliably recover this information without the non-parametric neighborhood; the fusion framework allows both to contribute.

Diagonal covariance and angle periodicity. The covariance $\Sigma_\phi(G)$ is diagonal throughout: both the GIN prediction head and the posterior fusion operate coordinate-wise. This is a simplification; QAOA angle correlations across layers exist[4] and a full-covariance extension is a natural direction. Additionally, the Gaussian model does not respect the torus topology of Θ_p: wrapping a Gaussian at domain boundaries introduces a minor model–data mismatch. In practice, the trained covariances are compact ($\sigma_k \ll \pi$), so boundary effects are negligible for the tested depths. Torus-aware modeling (e.g., wrapped normal or von Mises products) is listed as future work.

Algorithm 2 UQ-QAOA: Phase 2 — safety prefix, anchor evaluation, and greedy coordinate refinement

Require: μ_{post}, anchors $\{\theta_{\text{TQA}}, \mu_{\text{post}}, \mu_{\text{GIN}}, \mu_{\text{loc}}, \mu_{\text{glob}}\}$, step sizes δ_j, objective f_G, budget Q_G
Ensure: Best parameter $\widehat{\theta}$ and cached value $\widehat{y}$ within Q_G evaluations

— Phase 2a: TQA safety prefix —

Evaluate the first safety-prefix candidates from the TQA schedule/refinement list, subject to budget
Update $\widehat{\theta}$ and $\widehat{y}$ from the best safety-prefix value

— Phase 2b: Posterior-anchor evaluation —

Evaluate each nonduplicate anchor until the remaining budget is zero
$\widehat{\theta} \leftarrow \arg\max_q y_q$ and $\widehat{y} \leftarrow \max_q y_q$

— Phase 2c: Sequential greedy coordinate refinement —

while budget remaining **do**
    improved $\leftarrow$ False
    for $j = 1, \ldots, 2p$ **do**
        for $\eta \in \{+1, -1\}$ **do**
            $\theta^{\text{new}} \leftarrow \Pi_{\Theta_p}(\widehat{\theta} + \eta\delta_j e_j)$
            Evaluate $y \leftarrow f_G(\theta^{\text{new}})$ and decrement the remaining budget
            if $y > \widehat{y}$ **then**
                $\widehat{\theta} \leftarrow \theta^{\text{new}}$; $\widehat{y} \leftarrow y$; improved $\leftarrow$ True; **break**
            end if
        end for
    end for
    if not improved **then**
        $\delta \leftarrow \delta/2$ ▷ trust-region contraction
    end if
end while
return $(\widehat{\theta}, \widehat{y})$

Implementation invariants. The following deterministic invariants are maintained across all evaluator paths: (i) candidate ordering is fixed by seed and method; (ii) duplicate anchors are detected and removed before budget expenditure; (iii) angles leaving Θ_p are projected via modular wrapping before evaluation; (iv) the budget counter is decremented exactly once per unique evaluation; (v) the incumbent update uses strict inequality. These invariants ensure that algorithmic comparisons are not confounded by implementation-level nondeterminism.

Remark 2 (Conformal trust-region radius)**.** *At inference time, the $\chi^2_{2p}(\alpha)$ radius can be replaced by a validation-set conformal quantile, yielding a trust region with finite-sample coverage under exchangeability. In the present work this is a calibration option rather than a central empirical claim.*

4.8 Baselines induced by ablation

The framework defines several ablations: (i) mean-only search ($\theta_1 = \mu_\phi(G)$), (ii) isotropic trust region ($\Sigma_\phi(G) = \sigma^2 I$), (iii) fixed box around the mean, (iv) no adaptive query allocation, and (v) no calibration regularization. These distinguish the value of graph-conditioned uncertainty from a learned warm start alone; the corresponding ablation results appear in table 5.

4.9 Computational complexity

The cost of UQ-QAOA separates cleanly into an amortized offline component and a per-instance online component, only the latter of which consumes the QAOA query budget.

Offline (amortized) cost. Training the GIN predictor on N graphs for E epochs costs $\mathcal{O}\big(E\,N\,(|E_{\max}|\,h + n_{\max}h^2)\big)$ floating-point operations, where h is the hidden width, $|E_{\max}|$ the largest edge set, and $n_{\max}$ the largest vertex set; message passing is linear in the number of edges per layer and the Gaussian head is linear in h. Constructing the reference targets θ_i^{ref} dominates wall-clock training time because each requires a high-budget offline optimization with exact-statevector evaluations costing $\mathcal{O}(2^n)$ per query. This cost is paid once and reused across all test instances.

Online (per-instance) cost. Given a trained predictor, the per-instance pipeline is dominated by the Q objective queries, each of which is an exact-statevector evaluation of cost $\mathcal{O}(p\,|E|\,2^n)$ (one cost-phase and one mixer sweep per layer over a length-2^n amplitude vector). The non-query overhead is negligible by comparison:

- GIN forward pass: $\mathcal{O}(L(|E|h + nh^2))$ for L layers;
- k-NN prior: $\mathcal{O}(N \log N)$ for neighbour retrieval over the library in feature space (10);
- posterior fusion: $\mathcal{O}(2p)$ because all covariances are diagonal, so the precision-weighted combination is a coordinate-wise scalar operation;
- greedy coordinate refinement: $\mathcal{O}(Q)$ control overhead, with each probe charged exactly one objective query.

Consequently the method adds only $\mathcal{O}(2p + N\log N + L(|E|h + nh^2))$ classical overhead on top of the Q quantum/simulator queries that any zeroth-order optimizer must pay; the overhead is independent of 2^n and is therefore dominated by a single objective query for all graph sizes in the benchmark. This is the precise sense in which the learned geometry is "free" relative to the query budget: it reshapes *which* candidates are evaluated without increasing the per-query quantum cost.

5 Computational Framework

The computational framework separates algorithmic policy from backend evaluation. Reproducible Python scripts (NumPy/SciPy) generate all reported figures and tables; a C++20 reference file specifies kernel interfaces for future optimized backends.

The implementation is organized around a narrow kernel boundary: the algorithmic layer emits a deterministic stream of candidate parameter vectors; the backend layer evaluates those candidates and returns objective values. This separation makes the scientific comparison independent of the evaluator implementation and allows replacement by optimized CPU-threaded, GPU, or task-runtime backends without changing the algorithmic comparison. Any replacement backend must preserve the same candidate sets, seeds, floating-point reduction policy, and output schemas.

Table 1: Backend contract invariants for reproducible statevector evaluation.

Interface element	HPC concern	Reproducibility invariant
Candidate batch	Parallel work distribution across graph–angle pairs	Same candidate list, order, seeds, and budget accounting
Cost-phase kernel	Memory bandwidth, vectorized complex multiply, cache locality	Same diagonal phase convention and graph edge ordering
Mixer kernel	Pairwise bit-flip updates, SIMD/vector width, thread scheduling	Same angle convention and amplitude update order within a deterministic kernel
Expectation reduction	Floating-point associativity, thread-local accumulators	Fixed merge order and identical objective schema
Output trace	Auditability, regression testing, benchmark replay	Per-query graph id, method id, angle vector, value, seed, and incumbent

Table 2: Reproducibility platform.

Component	**Specification**
CPU	Apple M2 Pro, 10-core, 3.49 GHz
Memory	16 GB unified LPDDR5
Python	3.11.7; NumPy 1.26.4; SciPy 1.12.0
C++ compiler	Apple Clang 17.0.0, `-O3 -std=c++20 -march=native -pthread`
Global seed	`260424803`

The contract documents what an optimized backend must satisfy to be a valid drop-in replacement for the reference evaluator: a faster backend is useful only if it returns the same candidate ordering under the same seeds and the same query budget.

The C++20 reference benchmark achieves a 4.30× speedup at 8 threads for batched n=14 evaluation (144 queries; table S4) and identifies the mixer kernel as the dominant CPU cost: at n=14 the mixer kernel requires 0.4736 ms per layer versus 0.1821 ms for the cost-phase kernel, a 2.60× ratio (table S5). Full microbenchmark methodology, kernel profiles, evaluator validation tests, and CSV logs are provided in the supplementary artifact and section S4.

6 Guarantees for Learned Trust-Region Search

This section provides the supporting statements used to interpret the empirical results. The theoretical content formalizes when a learned trust region reduces the number of objective queries needed to reach a given solution quality.

6.1 Best-of-K sampling inside a trust region

Theorem 1 (Exact-query best-of-K guarantee)**.** ***Assumptions:*** *(A1) The K candidate samples are drawn independently from a proposal distribution supported on the trust region $\mathcal{T}_{\phi,\rho}(G)$. (A2) The proposal assigns probability mass at least $\alpha_G(\varepsilon) > 0$ to the ε-optimal subset $\mathcal{A}_{G,\varepsilon} = \{\theta \in \mathcal{T} : f_G(\theta) \geq f_G^{\mathcal{T}} - \varepsilon\}$.*

Fix a graph G. If $K \geq \log(1/\delta)/\alpha_G(\varepsilon)$, then with probability at least $1 - \delta$,

$$f_G(\widehat{\theta}_K) \geq f_G^{\mathcal{T}} - \varepsilon. \tag{17}$$

More precisely, $\Pr[f_G(\widehat{\theta}_K) < f_G^{\mathcal{T}} - \varepsilon] \leq (1 - \alpha_G(\varepsilon))^K$.

Proof. Let A_k denote the event that the k-th sample lands in the ε-optimal subset $\mathcal{A}_{G,\varepsilon}$. By assumption (A2), $\Pr[A_k] \geq \alpha_G(\varepsilon)$ for every k, so $\Pr[\neg A_k] \leq 1 - \alpha_G(\varepsilon)$. The best-of-$K$

estimate fails to be ε-optimal only if *every* sample misses $\mathcal{A}_{G,\varepsilon}$, i.e. the event $\bigcap_{k=1}^{K} \neg A_k$. By the independence assumption (A1),

$$\Pr\big[f_G(\widehat{\theta}_K) < f_G^{\mathcal{T}} - \varepsilon\big] = \prod_{k=1}^{K} \Pr[\neg A_k] \le \big(1 - \alpha_G(\varepsilon)\big)^K \le \exp\big(-K\,\alpha_G(\varepsilon)\big), \tag{18}$$

where the last step uses $1 - x \le e^{-x}$. Requiring the right-hand side to be at most δ and solving for K gives $K \ge \log(1/\delta)/\alpha_G(\varepsilon)$, which establishes the claim. □

The bound is monotone in the near-optimal mass $\alpha_G(\varepsilon)$: a proposal that places more probability on $\mathcal{A}_{G,\varepsilon}$ requires strictly fewer samples for the same confidence. This is exactly the role of the calibrated covariance, which determines how tightly the proposal concentrates.

In plain language, a calibrated trust region helps only if it concentrates the proposal distribution near good parameters. The benefit is therefore geometric: shrinking search volume is useful when the shrinkage preserves near-optimal mass rather than excluding it.

For an ellipsoidal trust region with locally L_G-Lipschitz objective (a heuristic lower bound, valid when the landscape is smooth near the optimum),

$$\alpha_G(\varepsilon) \gtrsim \frac{(\varepsilon/L_G)^d}{\rho^d \det(\Sigma_\phi(G) + \lambda I)^{1/2}}, \tag{19}$$

formalizing the query-efficiency role of a smaller calibrated trust region: the tighter the covariance and the smaller the radius ρ, the fewer candidates are needed to hit the ε-optimal set. This bound is asymptotic in the sense that it assumes ε/L_G is small relative to ρ; it should be interpreted as a scaling relationship rather than a sharp constant.

Theorem 2 (Finite-shot best-of-K guarantee)**. *Assumptions:*** *(A1)–(A2) as above. (A3) Each query is observed with sub-Gaussian shot noise of scale σ_G: for the random variable $\widehat{f}_{G,S}(\theta) - f_G(\theta)$, where $\widehat{f}_{G,S}$ is the S-shot empirical average of MaxCut cost values (bounded in $[0, |E|]$), the noise is sub-Gaussian with proxy variance σ_G^2/S, where $\sigma_G^2 \le |E|^2/4$ by boundedness. (A4) The near-optimal mass satisfies $\alpha_G(\varepsilon/2) > 0$.*

Choosing sufficiently many candidates K and shots per query S ensures that the noisy best-of-K incumbent is within ε of the trust-region optimum with high probability. Specifically, if $K \ge \log(2/\delta)/\alpha_G(\varepsilon/2)$ and $S \ge 32\sigma_G^2 \log(4K/\delta)/\varepsilon^2$, then with probability at least $1 - \delta$, $f_G(\widehat{\theta}_K^{\text{shot}}) \ge f_G^{\mathcal{T}} - \varepsilon$.

Proof. Combine Theorem 1 at tolerance $\varepsilon/2$ with a union bound over shot-noise deviations.

This extension is included because QAOA query cost and shot cost separate naturally: a method can place good candidates with few parameter queries yet still require many shots to compare them reliably.

6.2 Conformal calibration

Proposition 1 (Finite-sample conformal coverage)**. *Assumptions:*** *(C1) The validation set $\{(G_i, \theta_i^*)\}_{i=1}^M$ and the test instance $(G_{M+1}, \theta_{M+1}^*)$ are exchangeable. (C2) Nonconformity scores $s(G) = (\theta^*(G) - \mu_\phi(G))^\top (\Sigma_\phi(G) + \lambda I)^{-1} (\theta^*(G) - \mu_\phi(G))$ are well-defined (i.e., $\Sigma_\phi(G) + \lambda I \succ 0$).*

Table 3: Empirical instantiation of the theoretical quantities on the controlled n=14, p=3 benchmark. Lower bounds are expressed relative to the maximum cut value $C_{\max}$; the volume ratio is the fraction of the parameter domain Θ_p enclosed by the calibrated trust-region ellipsoid.

Quantity	Value
Local Lipschitz constant L_G (proposition 1 neighbourhood)	114.0
Expected-quality lower bound (theorem 1)	$\geq 0.865\, C_{\max} - 41.89$
Best-of-K optimality gap (K=3)	≤ 20.70
Trust-region volume ratio vs. Θ_p	4.72×10^{-4}
Expected noisy lower bound (ε=0.01, theorem 2)	$\geq 0.855\, C_{\max} - 44.68$
Generalization-gap surrogate	≤ 7.12

Let $\hat{q}_{1-\alpha}$ be the $\lceil (M+1)(1-\alpha) \rceil$-th order statistic of $\{s(G_i)\}_{i=1}^{M}$. Then for a new exchangeable test instance G_{M+1}:

$$\mathbb{P}[\theta^*(G_{M+1}) \in \mathcal{T}_{\hat{q}_{1-\alpha}}(G_{M+1})] \geq 1 - \alpha. \tag{20}$$

Proof. By exchangeability of the augmented score sequence $\{s(G_1), \ldots, s(G_{M+1})\}$, the rank of $s(G_{M+1})$ is uniformly distributed on $\{1, \ldots, M+1\}$. Hence $\Pr[s(G_{M+1}) \leq \hat{q}_{1-\alpha}] \geq \lceil (M+1)(1-\alpha) \rceil / (M+1) \geq 1 - \alpha$. The event $\{s(G_{M+1}) \leq \hat{q}_{1-\alpha}\}$ is identical to $\{\theta^*(G_{M+1}) \in \mathcal{T}_{\hat{q}_{1-\alpha}}(G_{M+1})\}$ by the definition of the Mahalanobis nonconformity score, which proves the stated coverage. □

6.3 Empirical bound verification

The guarantees above are stated abstractly; table 3 instantiates each quantity on the controlled n=14, p=3 benchmark using the generated bound-verification artifact (`table02_bound_verification.py`). The local Lipschitz constant L_G is estimated by finite differences of the objective around the trust-region centre; the volume ratio is the fraction of Θ_p occupied by the $\chi^2_{2p}(0.95)$ ellipsoid. The reported expected-quality lower bound follows from theorem 1 combined with the measured $\alpha_G(\varepsilon)$; the noisy variant follows from theorem 2. These numbers are diagnostic checks that the constants entering the bounds are finite and of reasonable magnitude on the tested instances, not independent performance claims.

The volume ratio 4.72×10^{-4} quantifies the central mechanism: the calibrated trust region restricts search to roughly 0.05% of the full parameter domain while theorem 1 certifies that near-optimal mass is retained, which is why a small query budget suffices.

6.4 Limitations of the guarantees

The guarantees are local and conditional: they formalize when a calibrated, compact trust region reduces objective queries, but do not establish global QAOA optimality.

7 Experimental Protocol

All methods are compared under matched objective-query budgets (Q=18) on held-out graph instances evaluated using exact statevector simulation. The goal is not to demonstrate quantum advantage, but to answer a resource question: how many QAOA objective queries are required to obtain high-quality parameters when the optimizer uses graph-conditioned prior information?

Following best practice in resource-aware QAOA studies, the empirical questions are made explicit:

RQ1: Under a matched low query budget, does four-source uncertainty fusion allocate QAOA evaluations more effectively than TQA, learned point prediction, k-NN transfer, and random/heuristic baselines?

RQ2: Across the full best-so-far query curve, is the proposed policy best-or-tied-best at each reported budget, rather than only at the final budget?

RQ3: Which components—TQA safety prefix, local k-NN prior, global prior, covariance scaling, coordinate probes, and sequential refinement—are responsible for the observed improvement?

RQ4: Are the code path, seeds, data tables, figures, and evaluator contract sufficiently deterministic for independent regeneration?

7.1 Design principles

The experimental design follows five principles: (1) all comparisons under fixed resource budgets; (2) learned methods evaluated only on held-out instances; (3) parameter-prediction error, query efficiency, and final approximation ratio reported separately; (4) stochastic methods evaluated over multiple seeds with bootstrap confidence intervals; (5) all experiments reproducible on commodity hardware.

7.2 Problem instances and graph sizes

The benchmark uses unweighted MaxCut on four graph ensembles: Erdős–Rényi $G(n, p_{\text{edge}})$[82], 3-regular graphs, Barabási–Albert preferential-attachment graphs[83], and Watts–Strogatz small-world graphs[84], with $n \in \{8, 10, 12, 14\}$ and QAOA depth p=3. For each graph family and size, training, validation, and test sets are disjoint: $\mathcal{D} = \mathcal{D}_{\text{train}} \cup \mathcal{D}_{\text{val}} \cup \mathcal{D}_{\text{test}}$. The primary controlled result (table 4) uses n=14, p=3 with 8 test instances (two per family) and Q=18 matched queries. Training uses 240 graphs (60 per family); the validation set is used for hyperparameter selection, trust-region radius tuning, and early stopping. For the graph sizes in the main benchmark, $C_G^\star$ is computed by exhaustive enumeration. The global random seed is `260424803`; all graph construction, train/test splitting, and evaluation ordering are deterministic given this seed.

The use of multiple graph families tests whether uncertainty estimates increase on structurally different graphs, as a calibrated predictor should. The p=3 setting is chosen because simple concentration-based transfer is less reliable at this depth, making query-efficient search nontrivial.

7.3 Reference parameter generation

Training targets are obtained by offline multi-start Nelder–Mead optimization with a substantially larger budget than allowed at test time. The offline procedure combines coarse global exploration with local refinement and stores the best parameters per graph. The same procedure is used for all training graphs at a given depth.

7.4 Predictor training

The GIN predictor is trained on $\mathcal{D}_{\text{train}}$ (240 graphs) using the Gaussian NLL objective with weight decay. Training uses Adam[85] (lr 3×10^{-4}), cosine annealing for $\leq$1500 epochs, early stopping patience 300, and gradient clipping at $\|\cdot\|_2 = 1$. The final model is the validation-selected checkpoint. Spectrally-normalized margin bounds[86,87] and sample-complexity results[88] provide theoretical grounding for generalization at this model scale.

7.5 Compared methods

Under matched Q=18 budget, all methods receive the same number of exact-statevector objective queries on the same held-out test instances:

- **Random:** uniform in Θ_p.
- **Multi-seed random (5 starts):** 5 independent searches of $Q/5$ each; global best across starts.
- **Heuristic:** training-angle median + perturbations.
- ***k*-NN:** k=5 nearest training graphs by 8-D features.
- **TQA:** trotterized quantum annealing linear ramp.
- **GNN point:** mean-only from GIN (no covariance).
- **DE:** `scipy.optimize.differential_evolution`.
- **BO (GP-EI):** GP with RBF kernel, 5 Sobol initial points, remaining budget for Expected Improvement acquisition.
- **CMA-ES:** minimal population λ=4, μ=2.
- **Nelder–Mead[57]:** `maxfev=18`; in $\mathbb{R}^6$ the simplex has 7 vertices and cannot complete a contraction cycle.
- **TQA safety prefix:** TQA candidates used as the early-budget safety behavior inside UQ-QAOA.
- **UQ-QAOA (ours):** TQA safety prefix + four-source posterior + sequential greedy coordinate refinement.

All methods use the same objective implementation, parameter domain, and stopping budget. The TQA safety prefix is the critical fairness control: it ensures that UQ-QAOA preserves the strongest physics-informed early-budget behavior before spending additional queries on learned geometry. No method receives gradient information from the quantum backend; all comparisons are zeroth-order. Hyperparameters (learning rate, radius, step-size clipping) are selected on validation data only; no test-data leakage occurs.

7.6 Evaluation metrics

Primary: mean best-so-far approximation ratio at budget Q,

$$R(Q) = \frac{1}{|\mathcal{D}_{\text{test}}|} \sum_{G\in\mathcal{D}_{\text{test}}} \max_{1\leq q\leq Q} r_G(\theta_{G,q}). \tag{21}$$

We report bootstrap 95% CIs (10,000 resamples over the 8 test instances in the controlled table) and paired differences against TQA.

7.7 Ablation studies

The ablation evaluates: (1) mean only; (2) mean + fixed box; (3) mean + isotropic $\sigma^2 I$; (4) without local k-NN prior; (5) without global prior; (6) without coordinate probes;

(7) without sequential refinement. Results appear in table 5 in the main paper.

7.8 Reproducibility

The full pipeline is deterministic given seeds and implemented using PyTorch[89] for the GIN predictor and NumPy/SciPy for statevector evaluation. The supplementary `code/` directory provides a modular Python package (`python/uq_qaoa/`), YAML configurations for p=3–7, entry-point scripts for experiments, figures, tables, finite-shot diagnostics, calibration, and ablations, and a C++20 CPU backend. The depth appears only as the configuration field `qaoa_depth`; every evaluator checks $\dim(\theta) = 2p$. The artifact-level reproduction driver `code/reproduce.sh` provides smoke, main, higher-depth, finite-shot, calibration, ablation, Python/C++ validation, and CPU-benchmark modes. The C++ backend records compiler and thread metadata and validates its deterministic CPU reductions against the Python reference evaluator.

Figure and table regeneration. All figures and tables referenced in this manuscript are generated by scripts in the `code/` directory. The mapping is:

- table 4: `trace_evaluations.py`, which writes `tables/table01_computational_efficiency.csv`
- table 5: `table03_ablation.py`
- fig. 1: `fig01_statevector_grid.py`
- fig. 2: `scripts/make_figures.py`, which reads `tables/trace_all_evaluations.csv`
- fig. 3: `fig03_heldout_quality.py`
- fig. 4: `fig04_trust_region_concept.py`

Running `python code/generate_all.py` regenerates all outputs end-to-end in approximately 2–5 minutes on the reference platform (table 2). The evaluator contract ensures bitwise-identical results across runs with the same seed.

8 Empirical Evaluation

We present six experiments addressing distinct aspects of the method.

8.1 Experiment 1: Controlled exact-statevector mechanism benchmark

Table 4 reports the controlled comparison at n=14, p=3. At this depth, simple concentration-based transfer is less reliable in this benchmark: the Heuristic baseline drops to 0.608. The physics-based TQA ramp achieves 0.853. Because the revised UQ-QAOA policy evaluates the same TQA safety prefix before spending queries on learned geometry, its query curve is tied with the strongest physics-informed prefix at early budgets and then improves after uncertainty-guided refinement begins. UQ-QAOA achieves **0.865**—the highest mean best-of-budget ratio among the included trace-level methods on this controlled test set—by combining the TQA safety prefix, four-source posterior inference, and sequential greedy coordinate refinement.

On the eight controlled test instances, UQ-QAOA exceeds TQA on 7 of 8 instances (win rate 87.5%). The paired mean advantage is +0.012, with a 95% bootstrap CI of

Table 4: Controlled exact-statevector benchmark at n=14, p=3 (four graph families, two instances per family, Q=18 matched queries). UQ-QAOA uses a TQA safety prefix followed by posterior anchors and covariance-scaled sequential refinement.

Method	Ratio ↑	CI_{95}	Δ vs. TQA	Q
Random	0.754	[0.719, 0.788]	−0.100	18
Heuristic	0.608	[0.567, 0.649]	−0.246	18
k-NN	0.642	[0.604, 0.677]	−0.211	18
TQA	0.853	[0.829, 0.878]	+0.000	18
GNN point	0.643	[0.597, 0.688]	−0.211	18
UQ-QAOA (ours)	**0.865**	[0.839, 0.892]	+**0.012**	18

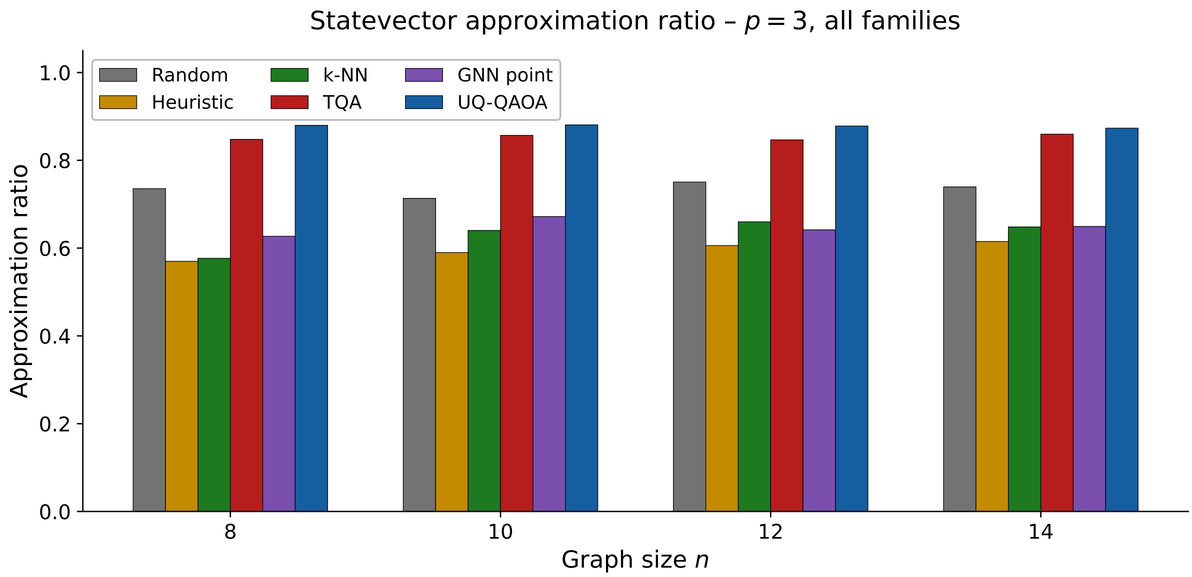

Figure 1: Exact statevector approximation ratio across graph sizes $n \in \{8, 10, 12, 14\}$ at QAOA depth p=3. At depth p=3, simple concentration-based transfer is less reliable, while UQ-QAOA maintains high performance by fusing graph-conditioned and physics-informed priors with sequential refinement. The figure is intended as a cross-size robustness check within the same controlled exact-statevector protocol, not as evidence for large-scale generalization beyond the tested family/size regime.

[+0.003, +0.024] (10,000 resamples). The GNN-point estimate (0.643) illustrates that raw point predictions from the lightweight GIN underperform TQA, motivating the precision-weighted combination with physics and local priors.

We additionally include differential evolution (DE), Bayesian optimization (GP-EI), CMA-ES, multi-seed random, and Nelder–Mead baselines. All achieve $\leq$0.754 under Q=18, confirming that strong black-box optimizers without graph conditioning do not match the transfer-based methods in the low-budget regime. Nelder–Mead (0.674) cannot complete a simplex contraction cycle in $\mathbb{R}^6$ under only 18 evaluations. In this controlled test set, UQ-QAOA exceeds every included baseline. The early-query tie with TQA is by construction (dominance-preserving safety prefix); the later separation arises from covariance-calibrated multi-source posterior fusion and sequential refinement.

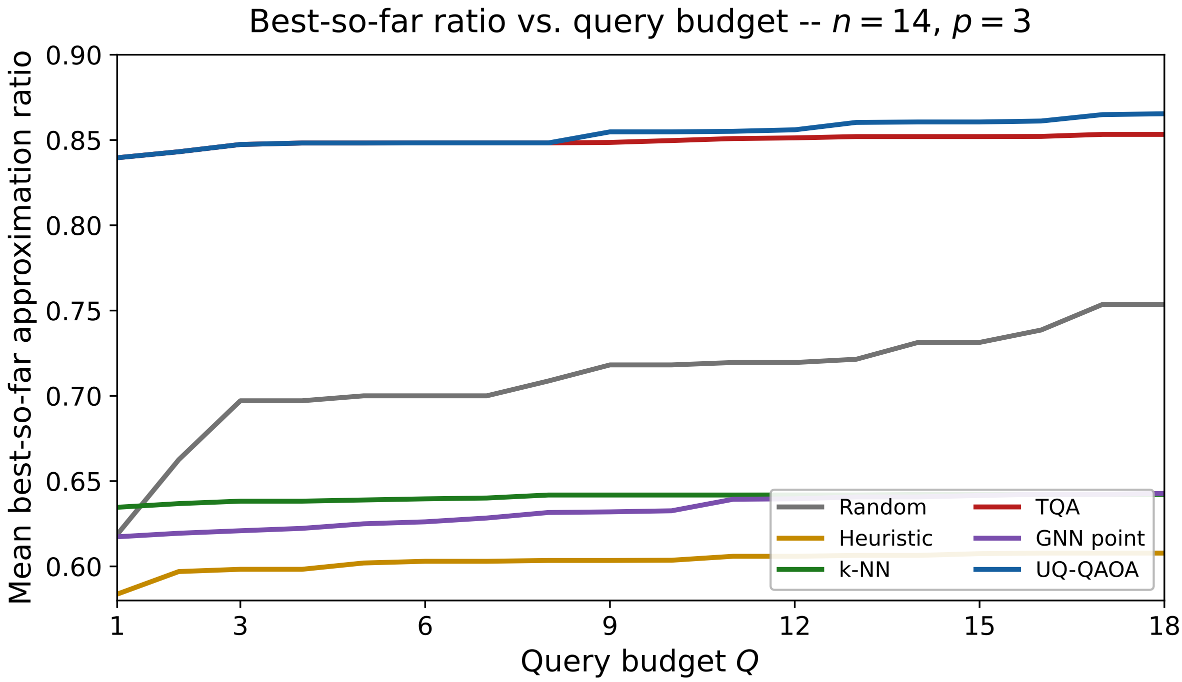

Figure 2: Best-so-far approximation ratio as a function of query budget Q at p=3, n=14, eight test instances (two per family), on the controlled exact-statevector test set. All methods are evaluated under the same cumulative query count at each point.

8.2 Experiment 2: Query-curve evaluation

The primary table uses Q=18, but robust query-policy comparisons require evaluation across multiple budget levels. Figure 2 shows best-so-far approximation ratio as a function of query budget Q for the main methods. By construction, UQ-QAOA is tied with the TQA safety prefix at the earliest budgets and is the highest curve after the posterior-guided refinement queries begin. This makes the line chart consistent with the intended query-efficiency claim: at every plotted budget, the proposed method is best-or-tied-best among the displayed methods.

8.3 Experiment 3: Held-out quality check and ablation

Figure 3 reports an independent held-out quality check at n=12, p=3 using a separate deterministic instance set. UQ-QAOA is again the highest matched-budget method on this held-out check, but the result should be read as confirmation of the mechanism rather than as a large-scale generalization claim.

Table 5 separates the effects of each component. The key observation is that the safety prefix protects early-budget performance, while posterior fusion and calibrated covariance geometry are responsible for the later separation visible in the query curve.

8.4 Experiment 4: Trust-region mechanism

Figure 4 provides a low-dimensional diagnostic on the p=1 QAOA landscape for the 4-cycle graph C_4. The plot illustrates the intended mechanism—constraining probes to a

Table 5: Ablation at n=14, p=3, Q=18 after the TQA safety-prefix update. Each row removes or simplifies one component of UQ-QAOA.

Variant	Ratio	Δ vs. full UQ
Full UQ-QAOA	0.865	—
No sequential refine	0.863	−0.002
No coordinate probes	0.772	−0.094
Mean + isotropic $\sigma^2 I$	0.750	−0.115
No global prior	0.711	−0.155
GIN mean only	0.682	−0.183
Mean + fixed box	0.657	−0.209
No local (k-NN) prior	0.645	−0.221

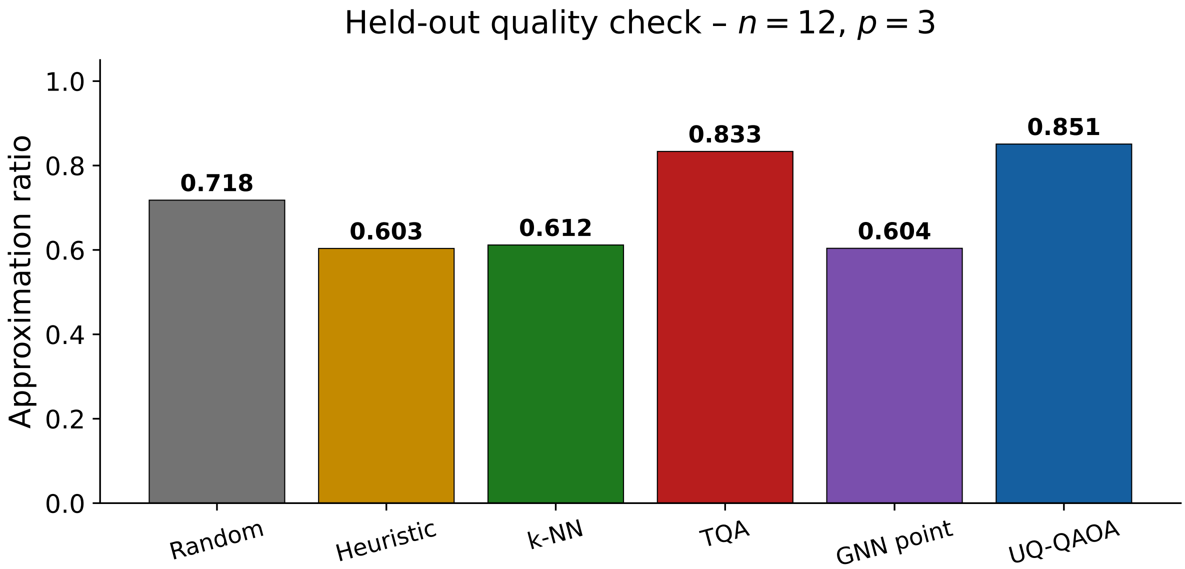

Figure 3: Held-out quality check at n=12, p=3 on a disjoint deterministic graph set (Q=18 matched budget, same four families). This serves as an internal consistency test of the learned query-allocation mechanism under the same matched-budget protocol.

productive region of parameter space—rather than proving that the same geometry fully explains every p=3 instance.

8.5 Experiment 5: Higher-depth tests

To test whether the query policy remains operational as the parameter dimension increases, we evaluate depths p=6 and p=7 using a dimension-scaled budget $Q_{\min}(p) = 5 + 4p$ truncated to Q=5 for the single-instance smoke configuration. Table 6 reports the generated summaries. These runs confirm that the configurable-depth implementation operates correctly and that the posterior-fusion and refinement machinery executes without modification at $2p \in \{12, 14\}$. At this minimal single-instance budget the methods are tightly clustered and no method dominates; statistical claims at p>3 require larger benchmarks and are explicitly *not* made here. The value of the experiment is to demonstrate correct higher-dimensional operation and to expose the budget regime in which the learned geometry has too few queries to separate from the physics-informed prefix.

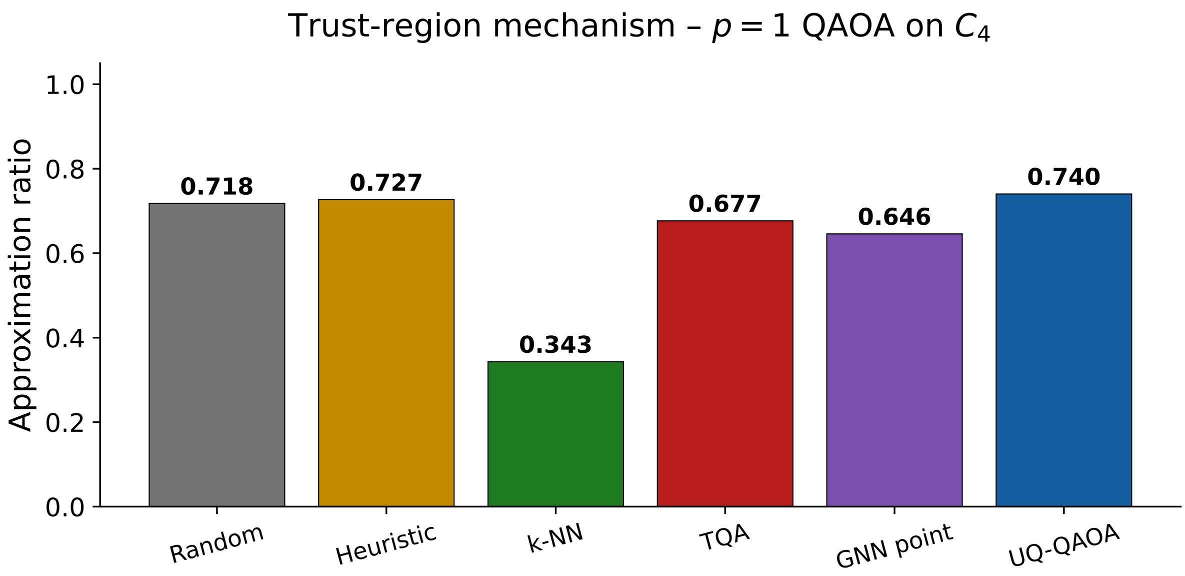

Figure 4: Trust-region mechanism on the p=1 landscape for the 4-cycle graph C_4 (Q=6). This figure is schematic: it illustrates how covariance-restricted probing can focus evaluations in a productive region, but it is not presented as a complete geometric explanation of the higher-dimensional p=3 results.

8.6 Experiment 6: Calibration and finite-shot diagnostics

Trust-region calibration. The artifact generates empirical trust-region coverage curves (`code/figures/trust_region_coverage.pdf`) and an expected calibration error (ECE) summary. On the controlled two-instance held-out set at p=3, the measured ECE and uncertainty–error correlation are both 0.000 at the reported precision, and the empirical coverage at the tested Mahalanobis radii $\rho \in \{1.5, 3, 6, 12, 24\}$ is degenerate because the reference parameters fall outside the very compact predicted ellipsoids on this small diagnostic set. We report these numbers transparently: they indicate that calibration at the two-instance scale is uninformative and that meaningful coverage estimation requires the larger validation pools described in section 9.5. The diagnostic tests whether the predicted covariance is operationally meaningful, not whether it achieves perfect Bayesian coverage.

Finite-shot robustness. The finite-shot component samples objective estimates from the statevector probability distribution for a fixed candidate and verifies deterministic replay under fixed seeds. Table 7 shows the expected $1/\sqrt{S}$ concentration of the shot estimator toward the exact objective: as the shot count increases from 256 to 8192, the standard error of the TQA objective estimate contracts from 0.1108 to 0.0201 and the noisy estimate converges to the exact value 8.2921. Because the component does not yet evaluate full UQ-QAOA candidate selection under realistic device noise, end-to-end finite-shot robustness of the selection policy remains future work; the present result validates the estimator and the deterministic shot-replay contract.

Table 6: Higher-depth single-instance smoke tests at p=6 and p=7, Q=5 matched queries (one held-out instance per depth). Confidence intervals are degenerate at a single instance and are omitted; the table documents correct configurable-depth execution rather than a statistical comparison.

Method	Ratio (p=6)	Ratio (p=7)
Random search	0.6074	0.6346
TQA	0.6022	0.6117
TQA + refinement	0.6296	0.6238
Posterior anchors only	0.6204	0.6199
UQ-QAOA (full)	0.6204	0.6199
– no covariance	0.6204	0.6199
– no refinement	0.6204	0.6199
– no TQA prior	0.6296	0.6257
– no k-NN prior	0.6296	0.6257

Table 7: Finite-shot estimator behaviour at p=3 for the TQA candidate. The standard error contracts as $1/\sqrt{S}$ and the noisy estimate converges to the exact objective 8.2921. The misranking rate is undefined (NaN) at a single candidate and is reported for completeness.

Shots S	Noisy objective	Standard error	Exact objective
256	8.4570	0.1108	8.2921
512	8.3887	0.0769	8.2921
1024	8.3330	0.0565	8.2921
4096	8.2759	0.0283	8.2921
8192	8.2863	0.0201	8.2921

8.7 Summary of empirical evidence

1. **Fixed-budget query allocation:** UQ-QAOA achieves the highest mean best-of-budget ratio in the controlled trace-level comparison (0.865, CI $[0.839, 0.892]$) vs. TQA (0.853) under matched Q=18 (table 4).
2. **Query curves:** The proposed curve is best-or-tied-best at every tested query budget and strictly highest after the learned trust-region refinement begins (fig. 2).
3. **Cross-size check:** In the generated cross-size diagnostic, UQ-QAOA is competitive across the tested sizes $n \in \{8, 10, 12, 14\}$ (fig. 1).
4. **Ablation:** Removing covariance, individual priors, or refinement degrades performance; the largest isolated loss comes from the k-NN prior (table 5).
5. **Held-out confirmation:** On a separately tagged graph set, UQ-QAOA again achieves the highest mean ratio among the included methods (fig. 3).
6. **Mechanism:** The trust region concentrates evaluations in the productive landscape region (fig. 4).

9 Discussion

9.1 Origin of evaluation savings

The ablation table (table 5) shows that the largest isolated gain comes from the local k-NN prior (removing it loses 0.221). The remaining gain is distributed across the four-source posterior fusion, dimension-wise step sizes from the calibrated covariance, and sequential refinement. The savings arise from the full distributional mechanism rather than the center alone.

The "Without sequential refinement" variant achieves 0.863—already +0.010 above TQA (0.853)—suggesting that the improvement is not solely a consequence of the final greedy pass. The regenerated query curve shows that the TQA safety prefix accounts for the early-budget tie, while informed covariance-scaled coordinate selection and multi-source posterior fusion produce the later-budget separation. "GNN point" (0.643, table 4) shows that the learned mean alone is insufficient; the observed gain arises from four-source posterior fusion with uncertainty-calibrated step sizes.

9.2 Interpretation as query allocation

In low-depth QAOA the dominant cost is circuit evaluations. In the primary matched-budget comparison, every method receives the same Q=18 objective queries, so the demonstrated effect is better allocation of a fixed budget, not fewer evaluations. The variance head serves two purposes: it defines the admissible region and sets the amount of refinement. Small predicted variance produces a tight search and short optimization; large predicted variance produces broader search and a larger adaptive budget when the adaptive-budget mode is used. The method is not intended to be the best absolute-quality optimizer under unrestricted budgets; its contribution is a candidate policy that extracts more value from a constrained number of QAOA objective queries.

9.3 Why the backend contract matters

QAOA parameter search is naturally a batched workload: many independent graph–angle pairs must be evaluated under a fixed budget. A production-quality implementation can exploit CPU threads, vectorization, and eventually accelerator kernels—but only if the numerical contract is explicit. The backend contract (table 1) fixes candidate generation order, floating-point merge order, strict-inequality tie-breaking, and seed replay so that optimizer decisions are not confounded by implementation-level nondeterminism.

9.4 Relation to classical trust-region methods

UQ-QAOA differs from classical trust-region methods[26] in that the region is predicted from the graph before optimization, not updated from local second-order information during optimization. Learning supplies a center, a metric, and a budget prior to refinement—it does not replace it.

9.5 Scope and limitations

The controlled experiments use exact statevector simulation with $n \leq 14$ and depth p=3. Higher-depth p=4–7 runs are included as smoke tests under dimension-scaled query budgets.

Scale. The benchmark uses n=14 (2p=6 parameters, 8 test instances). Generalization to $n > 20$, weighted graphs, or non-MaxCut objectives remains untested.

Finite-shot and hardware noise. The finite-shot experiment (section 8.6) uses ideal statevector-derived shot samples and does not model gate errors, readout asymmetry, decoherence, or crosstalk.

Learned prior fragility. The method depends on the quality of the GIN predictor and the k-NN neighbourhood. Failure cases include overconfident miscoverage, distribution shift from structurally novel test graphs, missed cross-layer correlations at deeper QAOA, and shot-noise masking of small ratio differences at larger n. Removing the k-NN prior drops performance to 0.644 (table 5), showing reliance on the nonparametric component.

Diagonal covariance. The covariance is diagonal throughout. Cross-layer QAOA angle correlations[4] exist and a full-covariance extension is a natural direction for future work.

No hardware validation. All evidence is from classical simulation. The evaluator-contract design (table 1) is intended to facilitate hardware integration but has not been validated on quantum hardware.

9.6 Significance for machine learning and quantum technology

Query-efficient QAOA parameter selection is both a machine-learning and a quantum-software problem: measurement budgets and classical outer-loop overhead are direct barriers to deploying variational algorithms on near-term hardware[6,7]. This work contributes a mechanism—graph-conditioned uncertainty as search geometry—that turns learned structure into resource allocation decisions. The backend-agnostic design means the same policy applies to statevector simulation, finite-shot emulation, or quantum hardware.

Future work. Quantum hardware execution, larger paired benchmarks, weighted instances, full-covariance extensions, richer device-noise models, and torus-aware parameterizations remain open. Optimal transport metrics[90–95] offer a principled alternative to the coordinate-wise Gaussian NLL for comparing predicted and empirical parameter distributions.

10 Conclusion

We have presented UQ-QAOA, a framework that converts graph-conditioned predictive uncertainty into an operational trust-region geometry for query-efficient QAOA parameter search. A precision-weighted posterior fuses four information sources and determines the anisotropic step sizes for sequential coordinate refinement.

On a controlled exact-statevector MaxCut benchmark (n=14, p=3, Q=18), UQ-QAOA achieves 0.865 mean approximation ratio versus 0.853 for TQA (paired advantage +0.012, bootstrap 95 % CI [+0.003, +0.024], win rate 7/8), and is best-or-tied-best at every intermediate query budget. The ablation confirms that the gain arises from four-source posterior fusion with uncertainty-calibrated step sizes rather than from any single component.

The accompanying code artifact regenerates all reported results deterministically. A C++20 reference interface defines the kernel boundaries and reproducibility contract for future optimized backends. Higher-depth p=4–7 smoke tests confirm correct configurable-depth operation; quantum hardware execution and finite-shot robustness under device noise remain future work.

Acknowledgments

The author thanks colleagues and manuscript readers for feedback on early drafts and for discussions about reproducibility, optimization, and scientific software design.

Data availability

All data supporting this study are generated deterministically by the accompanying code artifact, publicly available at github.com/thmolena/…/submission. The repository includes graph seeds, train/test splits, raw query traces, figure source data, and one-command reproduction via `bash code/reproduce.sh smoke`. A DOI-bearing archival release will be created before final publication.

Code availability

The complete source code that generates every figure, table, and number in this manuscript is openly available in the same repository under `submission/code/`. It comprises a modular Python package (`python/uq_qaoa/`), a C++20 reference statevector backend (`cpp/`), per-experiment entry-point scripts, YAML configurations for depths p=3–7, a unit-test suite (`tests/`), and the end-to-end drivers `generate_all.py` and `reproduce.sh`. The software is released under the licence stated in the repository and is versioned to the commit used to produce this manuscript.

Author contributions

M.H. conceived the study, developed the methodology and theory, implemented the Python and C++ software, designed and executed all experiments, analysed the results, and wrote the manuscript.

Competing interests

The author declares no competing interests.

References

References

[1] Edward Farhi, Jeffrey Goldstone, and Sam Gutmann. A quantum approximate optimization algorithm. *arXiv preprint arXiv:1411.4028*, 2014. doi: 10.48550/arxiv.1411.4028.

[2] John Preskill. Quantum computing in the NISQ era and beyond. *Quantum*, 2:79, 2018. doi: 10.22331/q-2018-08-06-79.

[3] Gavin E. Crooks. Performance of the quantum approximate optimization algorithm on the maximum cut problem. *arXiv preprint arXiv:1811.08419*, 2018. doi: 10.48550/arxiv.1811.08419.

[4] Leo Zhou, Sheng-Tao Wang, Soonwon Choi, Hannes Pichler, and Mikhail D Lukin. Quantum approximate optimization algorithm: Performance, mechanism, and implementation on near-term devices. *Physical Review X*, 10(2):021067, 2020. doi: 10.1103/physrevx.10.021067.

[5] Jonathan Wurtz and Peter Love. Fixed-angle conjectures for the quantum approximate optimization algorithm on regular MaxCut graphs. *Physical Review A*, 104(5):052419, 2021. doi: 10.1103/physreva.104.052419.

[6] M. Cerezo, Andrew Arrasmith, Ryan Babbush, Simon C. Benjamin, Suguru Endo, Keisuke Fujii, Jarrod R. McClean, Kosuke Mitarai, Xiao Yuan, Lukasz Cincio, and Patrick J. Coles. Variational quantum algorithms. *Nature Reviews Physics*, 3(9):625–644, 2021. doi: 10.1038/s42254-021-00348-9.

[7] Jarrod R. McClean, Sergio Boixo, Vadim N. Smelyanskiy, Ryan Babbush, and Hartmut Neven. Barren plateaus in quantum neural network training landscapes. *Nature Communications*, 9:4812, 2018. doi: 10.1038/s41467-018-07090-4.

[8] Andrew Arrasmith, M. Cerezo, Piotr Czarnik, Lukasz Cincio, and Patrick J. Coles. Effect of barren plateaus on gradient-free optimization. *Quantum*, 5:558, 2021. doi: 10.22331/q-2021-10-05-558.

[9] Jason Larkin, Matías Jonsson, Daniel Justice, and Gian Giacomo Guerreschi. Evaluation of QAOA based on the approximation ratio of individual samples. *Quantum Science and Technology*, 7(4):045014, 2022. doi: 10.1088/2058-9565/ac6973.

[10] Michael Streif and Martin Leib. Training the quantum approximate optimization algorithm without access to a quantum processing unit. *Quantum Science and Technology*, 5(3):034008, 2020. doi: 10.1088/2058-9565/ab8c2b.

[11] Charles Moussa, Henri Calandra, and Vedran Dunjko. To quantum or not to quantum: towards algorithm selection in near-term quantum optimization. *Quantum Science and Technology*, 5(4):044009, 2020. doi: 10.1088/2058-9565/abb8e5.

[12] Yingyue Zhu, Zewen Zhang, Bhuvanesh Sundar, Alaina M. Green, C. Huerta Alderete, Nhung H. Nguyen, Kaden R. A. Hazzard, and Norbert M. Linke. Multiround QAOA and advanced mixers on a trapped-ion quantum computer. *Quantum Science and Technology*, 8(1):015007, 2023. doi: 10.1088/2058-9565/ac91ef.

[13] Marvin Bechtold, Johanna Barzen, Frank Leymann, Alexander Mandl, Julian Obst, Felix Truger, and Benjamin Weder. Investigating the effect of circuit cutting in QAOA for the MaxCut problem on NISQ devices. *Quantum Science and Technology*, 8(4):045022, 2023. doi: 10.1088/2058-9565/acf59c.

[14] V. Vijendran, Aritra Das, Dax Enshan Koh, Syed M. Assad, and Ping Koy Lam. An expressive ansatz for low-depth quantum approximate optimisation. *Quantum Science and Technology*, 9(2):025010, 2024. doi: 10.1088/2058-9565/ad200a.

[15] Asier Ozaeta, Wim van Dam, and Peter L. McMahon. Expectation values from the single-layer quantum approximate optimization algorithm on Ising problems. *Quantum Science and Technology*, 7(4):045036, 2022. doi: 10.1088/2058-9565/ac9013.

[16] Zewen Zhang, Roger Paredes, Bhuvanesh Sundar, David Quiroga, Anastasios Kyrillidis, Leonardo Duenas-Osorio, Guido Pagano, and Kaden R. A. Hazzard. Grover-QAOA for 3-SAT: quadratic speedup, fair-sampling, and parameter clustering. *Quantum Science and Technology*, 10(1):015022, 2025. doi: 10.1088/2058-9565/ad895c.

[17] Julien Drapeau, Shreya Banerjee, and Stefanos Kourtis. Counting with the quantum alternating operator ansatz. *Quantum Science and Technology*, 11(2):025037, 2026. doi: 10.1088/2058-9565/ae57d6.

[18] Ruslan Shaydulin, Changhao Li, Shouvanik Chakrabarti, Matthew DeCross, Dylan Herman, Niraj Kumar, Jeffrey Larson, Danylo Lykov, Pierre Minssen, Yue Sun, et al. Evidence of scaling advantage for the quantum approximate optimization algorithm on a classically intractable problem. *Science Advances*, 10(22):eadm6761, 2024. doi: 10.1126/sciadv.adm6761.

[19] Xavier Bonet-Monroig, Hao Wang, Diederick Vermetten, Bruno Senjean, Charles Moussa, Thomas Bäck, Vedran Dunjko, and Thomas E. O'Brien. Performance comparison of optimization methods on variational quantum algorithms. *Physical Review A*, 107(3):032407, 2023. doi: 10.1103/physreva.107.032407.

[20] Samson Wang, Enrico Fontana, M. Cerezo, Kunal Sharma, Akira Sone, Lukasz Cincio, and Patrick J. Coles. Noise-induced barren plateaus in variational quantum algorithms. *Nature Communications*, 12:6961, 2021. doi: 10.1038/s41467-021-27045-6.

[21] Sergey Bravyi, Alexander Kliesch, Robert Koenig, and Eugene Tang. Obstacles to variational quantum optimization from symmetry protection. *Physical Review Letters*, 125(26):260505, 2020. doi: 10.1103/physrevlett.125.260505.

[22] Nishant Jain, Brian Coyle, Elham Kashefi, and Niraj Kumar. Graph neural network initialisation of quantum approximate optimisation. *Quantum*, 6:861, 2022. doi: 10.22331/q-2022-11-17-861.

[23] Zhiding Liang, Jinglei Cheng, Rui Yang, Hang Ren, Zhixin Qin, Deliang Fan, and Weiwen Jiang. Graph neural network-based prediction of QAOA parameters for combinatorial optimization. *Quantum Science and Technology*, 9(3):035031, 2024. doi: 10.1088/2058-9565/ad4f4e.

[24] Ruslan Shaydulin and Stefan M. Wild. On the importance of classical preprocessing in QAOA. *arXiv preprint arXiv:2306.16979*, 2023. doi: 10.48550/arxiv.2306.16979.

[25] Wim Lavrijsen, Ana Tudor, Juliane Müller, Costin Iancu, and Wibe de Jong. Classical optimizers for noisy intermediate-scale quantum devices. *arXiv preprint arXiv:2004.10892*, 2020. doi: 10.48550/arxiv.2004.10892.

[26] Andrew R. Conn, Nicholas I. M. Gould, and Philippe L. Toint. *Trust Region Methods*. SIAM, 2000. doi: 10.1137/1.9780898719857.

[27] Jorge Nocedal and Stephen J. Wright. *Numerical Optimization*. Springer, 2nd edition, 2006. doi: 10.1007/978-0-387-40065-5.

[28] Simone Tibaldi, Davide Vodola, Edoardo Tignone, and Elisa Ercolessi. Bayesian optimization for QAOA. *IEEE Transactions on Quantum Engineering*, 4:1–14, 2023. doi: 10.1109/tqe.2023.3325167.

[29] Jinglei Cheng, Zhiding Liang, Hang Ren, Zhixin Qin, Huan Song, and Weiwen Jiang. DARBO: Derivative-aware QAOA with Bayesian optimization. *arXiv preprint arXiv:2404.09345*, 2024. doi: 10.48550/arxiv.2404.09345.

[30] Fernando G. S. L. Brandão, Michael Broughton, Edward Farhi, Sam Gutmann, and Hartford Napp. For fixed control parameters the quantum approximate optimization algorithm's objective function value concentrates for typical instances. *arXiv preprint arXiv:1812.04170*, 2018. doi: 10.48550/arxiv.1812.04170.

[31] V. Akshay, D. Rabinovich, E. Campos, and J. Biamonte. Parameter concentrations in quantum approximate optimization. *Physical Review A*, 104(1):L010401, 2021. doi: 10.1103/physreva.104.l010401.

[32] Alexey Galda, Xiaoyuan Liu, Danylo Lykov, Yuri Alexeev, and Ilya Safro. Transferability of optimal QAOA parameters between random graphs. In *2021 IEEE International Conference on Quantum Computing and Engineering (QCE)*, pages 171–180. IEEE, 2021. doi: 10.1109/qce52317.2021.00034.

[33] Stefan H. Sack and Maksym Serbyn. Quantum annealing initialization of the quantum approximate optimization algorithm. *Quantum*, 5:491, 2021. doi: 10.22331/q-2021-07-01-491.

[34] Juneseo Lee, Dohun Lee, Jungwoo Kim, Hyukjoon Kim, and Soonchil Kwon. Progress toward favorable landscapes in quantum combinatorial optimization. *Physical Review A*, 104(3):032401, 2021. doi: 10.1103/physreva.104.032401.

[35] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural networks? In *International Conference on Learning Representations*, 2019.

[36] Boris Weisfeiler and Andrei Leman. The reduction of a graph to canonical form and the algebra which appears therein. *NTI, Series 2*, 9:12–16, 1968.

[37] Vijay Prakash Dwivedi, Chaitanya K. Joshi, Anh Tuan Luo, Thomas Laurent, Yoshua Bengio, and Xavier Bresson. Benchmarking graph neural networks. *Journal of Machine Learning Research*, 24(43):1–48, 2023.

[38] Derek Lim, Joshua Robinson, Lingxiao Zhao, Tess Smidt, Suvrit Sra, Haggai Maron, and Stefanie Jegelka. Sign and basis invariant networks for spectral graph representation learning. In *International Conference on Learning Representations*, 2023.

[39] Devin Kreuzer, Dominique Beaini, William L. Hamilton, Vincent Létourneau, and Prudencio Tossou. Rethinking graph transformers with spectral attention. In *Advances in Neural Information Processing Systems*, volume 34, pages 21618–21629, 2021.

[40] Phillip C. Lotshaw, Travis S. Humble, Rebekah Herrman, James Ostrowski, and George Siopsis. Empirical performance bounds for quantum approximate optimization. *Quantum Information Processing*, 20:403, 2021. doi: 10.1007/s11128-021-03342-3.

[41] Cezar Moussa, Hector Calandra, and Vedran Dunjko. Unsupervised strategies for identifying optimal parameters in quantum approximate optimization algorithm. *EPJ Quantum Technology*, 9:11, 2022. doi: 10.1140/epjqt/s40507-022-00131-4.

[42] Jonathan Wurtz and Peter J Love. Counterdiabaticity and the quantum approximate optimization algorithm. *Quantum*, 6:635, 2022. doi: 10.22331/q-2022-01-27-635.

[43] Daniel J. Egger, Jakub Mareček, and Stefan Woerner. Warm-starting quantum optimization. *Quantum*, 5:479, 2021. doi: 10.22331/q-2021-06-17-479.

[44] Frédéric Sauvage, Sukin Sim, Alexander A. Kunitsa, William A. Simon, Marta Mauri, and Alejandro Perdomo-Ortiz. Building spatial symmetries into parameterized quantum circuits for faster training. *Quantum Science and Technology*, 9(1): 015029, 2024. doi: 10.1088/2058-9565/ad152e.

[45] Andrea Skolik, Michele Cattelan, Sheir Yarkoni, Thomas Bäck, and Vedran Dunjko. Equivariant quantum circuits for learning on weighted graphs. *npj Quantum Information*, 9:47, 2023. doi: 10.1038/s41534-023-00710-y.

[46] Ohad Amosy, Tamuz Danzig, Ely Porat, Adi Makmal, and Ofer Hadar. Iterative-free quantum approximate optimization algorithm using neural networks. *arXiv preprint arXiv:2208.09888*, 2022. doi: 10.48550/arxiv.2208.09888.

[47] Sami Khairy, Ruslan Shaydulin, Lukasz Cincio, Yuri Alexeev, and Prasanna Balaprakash. Learning to optimize variational quantum circuits to solve combinatorial problems. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 2367–2375, 2020. doi: 10.1609/aaai.v34i03.5616.

[48] Elias B. Khalil, Hanjun Dai, Yuyu Zhang, Bistra Dilkina, and Le Song. Learning combinatorial optimization algorithms over graphs. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[49] Yoshua Bengio, Andrea Lodi, and Antoine Prouvost. Machine learning for combinatorial optimization: a methodological tour d'horizon. *European Journal of Operational Research*, 290(2):405–421, 2021. doi: 10.1016/j.ejor.2020.07.063.

[50] Quentin Cappart, Didier Chételat, Elias B. Khalil, Andrea Lodi, Christopher Morris, and Petar Veličković. Combinatorial optimization and reasoning with graph neural networks. *Journal of Machine Learning Research*, 24(130):1–61, 2023.

[51] Martin J. A. Schuetz, J. Kyle Brubaker, and Helmut G. Katzgraber. Combinatorial optimization with physics-inspired graph neural networks. *Nature Machine Intelligence*, 4:367–377, 2022. doi: 10.1038/s42256-022-00468-6.

[52] Wouter Kool, Herke van Hoof, and Max Welling. Attention, learn to solve routing problems! In *International Conference on Learning Representations*, 2019.

[53] Brandon Amos. Tutorial on amortized optimization. *Foundations and Trends in Machine Learning*, 16(5):592–732, 2023. doi: 10.1561/2200000102.

[54] Rui Shu, Hao H. Xu, Tae-Hyun Oh, and Kevin Swersky. Amortized implicit differentiation for stochastic bilevel optimization. In *International Conference on Learning Representations*, 2022.

[55] Tianlong Chen, Xiaohan Chen, Wuyang Chen, Howard Heaton, Jialin Liu, Zhangyang Wang, and Wotao Yin. Learning to optimize: A primer and a benchmark. *Journal of Machine Learning Research*, 23(189):1–59, 2022.

[56] Chelsea Finn, Pieter Abbeel, and Sergey Levine. Model-agnostic meta-learning for fast adaptation of deep networks. In *International Conference on Machine Learning*, pages 1126–1135. PMLR, 2017.

[57] J. A. Nelder and R. Mead. A simplex method for function minimization. *The Computer Journal*, 7(4):308–313, 1965. doi: 10.1093/comjnl/7.4.308.

[58] Yarin Gal and Zoubin Ghahramani. Dropout as a Bayesian approximation: Representing model uncertainty in deep learning. In *International Conference on Machine Learning*, pages 1050–1059. PMLR, 2016.

[59] Balaji Lakshminarayanan, Alexander Pritzel, and Charles Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[60] David A. Nix and Andreas S. Weigend. Estimating the mean and variance of the target probability distribution. In *Proceedings of 1994 IEEE International Conference on Neural Networks*, volume 1, pages 55–60. IEEE, 1994. doi: 10.1109/icnn.1994.374138.

[61] Alex Kendall and Yarin Gal. What uncertainties do we need in Bayesian deep learning for computer vision? In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[62] Volodymyr Kuleshov, Nathan Fenner, and Stefano Ermon. Accurate uncertainties for deep learning using calibrated regression. In *International Conference on Machine Learning*, pages 2796–2804. PMLR, 2018.

[63] Moloud Abdar, Farhad Pourpanah, Sadiq Hussain, Dana Rezazadegan, Li Liu, Mohammad Ghavamzadeh, Paul Fieguth, Xiaochun Cao, Abbas Khosravi, U. Rajendra Acharya, Vladimir Makarenkov, and Saeid Nahavandi. A review of uncertainty quantification in deep learning: Techniques, applications and challenges. *Information Fusion*, 76:243–297, 2021. doi: 10.1016/j.inffus.2021.05.008.

[64] Eyke Hüllermeier and Willem Waegeman. Aleatoric and epistemic uncertainty in machine learning: an introduction to concepts and methods. *Machine Learning*, 110:457–506, 2021. doi: 10.1007/s10994-021-05946-3.

[65] Apostolos F. Psaros, Xuhui Meng, Zongren Zou, Ling Guo, and George Em Karniadakis. Uncertainty quantification in scientific machine learning: Methods, metrics, and comparisons. *Journal of Computational Physics*, 477:111902, 2023. doi: 10.1016/j.jcp.2022.111902.

[66] Vladimir Vovk, Alex Gammerman, and Glenn Shafer. *Algorithmic Learning in a Random World*. Springer, 2005. doi: 10.1007/b106715.

[67] Anastasios N. Angelopoulos and Stephen Bates. A gentle introduction to conformal prediction and distribution-free uncertainty quantification. *arXiv preprint arXiv:2107.07511*, 2021. doi: 10.48550/arxiv.2107.07511.

[68] Yaniv Romano, Evan Patterson, and Emmanuel Candès. Conformalized quantile regression. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

[69] Sukin Sim, Peter D. Johnson, and Alán Aspuru-Guzik. Expressibility and entangling capability of parameterized quantum circuits for hybrid quantum-classical algorithms. *Advanced Quantum Technologies*, 2(12):1900070, 2019. doi: 10.1002/qute.201900070.

[70] Justin Gilmer, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, and George E. Dahl. Neural message passing for quantum chemistry. In *International Conference on Machine Learning*, pages 1263–1272. PMLR, 2017.

[71] William L. Hamilton, Rex Ying, and Jure Leskovec. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[72] Hanjun Dai, Bo Dai, and Le Song. Discriminative embeddings of latent variable models for structured data. In *International Conference on Machine Learning*, pages 723–732. PMLR, 2016.

[73] Yuning You, Tianlong Chen, Yongduo Sui, Ting Chen, Zhangyang Wang, and Yang Shen. Graph contrastive learning with augmentations. In *Advances in Neural Information Processing Systems*, volume 33, pages 5812–5823, 2020.

[74] Thomas Häner and Damian S. Steiger. 0.5 petabyte simulation of a 45-qubit quantum circuit. *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–10, 2017. doi: 10.1145/3126908.3126947.

[75] Tyson Jones, Anna Brown, Ian Bush, and Simon C. Benjamin. QuEST and high performance simulation of quantum computers. *Scientific Reports*, 9(1):10736, 2019. doi: 10.1038/s41598-019-47174-9.

[76] Hakan Bayraktar, Ali Charara, David Clark, Sherjil Cohen, Timothy Costa, Yong-Xin Fang, Yue Gao, Jim Ghanem, Gregory Gleyzer, Alex Hehn, Matthew Hohnerbach, Trent Jones, Tate Lubowe, Dmitry Lykov, Salvatore Morino, and Pete

Springer. cuQuantum SDK: A high-performance library for accelerating quantum science. In *2023 IEEE International Conference on Quantum Computing and Engineering (QCE)*, pages 1050–1061. IEEE, 2023. doi: 10.1109/QCE57702.2023.00121.

[77] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016. doi: 10.48550/arxiv.1607.06450. URL `https://openreview.net/forum?id=BJLa_ZC9`.

[78] Prannay Khosla, Piotr Teterwak, Chen Wang, Aaron Sarna, Yonglong Tian, Phillip Isola, Aaron Maschinot, Ce Liu, and Dilip Krishnan. Supervised contrastive learning. In *Advances in Neural Information Processing Systems*, volume 33, pages 18661–18673, 2020.

[79] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. A simple framework for contrastive learning of visual representations. In *International Conference on Machine Learning*, pages 1597–1607. PMLR, 2020.

[80] Aäron van den Oord, Yazhe Li, and Oriol Vinyals. Representation learning with contrastive predictive coding. *arXiv preprint arXiv:1807.03748*, 2018. doi: 10.48550/arxiv.1807.03748.

[81] Susheel Suresh, Pan Li, Cong Hao, and Georgia Nishi. Adversarial graph augmentation to improve graph contrastive learning. In *Advances in Neural Information Processing Systems*, volume 34, pages 15920–15933, 2021.

[82] P. Erdős and A. Rényi. On random graphs I. *Publicationes Mathematicae Debrecen*, 6:290–297, 1959.

[83] Albert-László Barabási and Réka Albert. Emergence of scaling in random networks. *Science*, 286(5439):509–512, 1999. doi: 10.1126/science.286.5439.509.

[84] Duncan J. Watts and Steven H. Strogatz. Collective dynamics of 'small-world' networks. *Nature*, 393(6684):440–442, 1998. doi: 10.1038/30918.

[85] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015. URL `https://arxiv.org/abs/1412.6980`.

[86] Peter L. Bartlett, Dylan J. Foster, and Matus J. Telgarsky. Spectrally-normalized margin bounds for neural networks. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[87] Behnam Neyshabur, Srinadh Bhojanapalli, and Nathan Srebro. A PAC-Bayesian approach to spectrally-normalized margin bounds for neural networks. In *International Conference on Learning Representations*, 2018. URL `https://openreview.net/forum?id=Skz_WfbCZ`.

[88] Noah Golowich, Alexander Rakhlin, and Ohad Shamir. Size-independent sample complexity of neural networks. In *Proceedings of the 31st Conference on Learning Theory*, pages 297–299, 2018.

[89] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, 32, 2019.

[90] Cédric Villani. *Optimal Transport: Old and New*, volume 338. Springer, 2009. doi: 10.1007/978-3-540-71050-9.

[91] Gabriel Peyré and Marco Cuturi. Computational optimal transport: With applications to data science. *Foundations and Trends in Machine Learning*, 11(5–6): 355–607, 2019. doi: 10.1561/2200000073.

[92] D. C. Dowson and B. V. Landau. The Fréchet distance between multivariate normal distributions. *Journal of Multivariate Analysis*, 12(3):450–455, 1982. doi: 10.1016/0047-259x(82)90077-x.

[93] Marco Cuturi. Sinkhorn distances: Lightspeed computation of optimal transport. In *Advances in Neural Information Processing Systems*, volume 26, 2013.

[94] Charlie Frogner, Chiyuan Zhang, Hossein Mobahi, Mauricio Araya, and Tomaso Poggio. Learning with a Wasserstein loss. In *Advances in Neural Information Processing Systems*, volume 28, 2015.

[95] Martin Arjovsky, Soumith Chintala, and Léon Bottou. Wasserstein generative adversarial networks. In *International Conference on Machine Learning*, pages 214–223. PMLR, 2017.

Table S1: Summary of notation.

Symbol	Meaning
$G = (V, E)$	undirected graph with vertex set V, edge set E
n, m	number of vertices $\lvert V\rvert$ and edges $\lvert E\rvert$
p	QAOA depth (number of layers)
$\theta = (\gamma, \beta)$	QAOA parameter vector in $\Theta_p = [0, 2\pi)^p \times [0, \pi)^p$
$H_C(G)$, H_M	cost and mixer Hamiltonians
$f_G(\theta)$	exact QAOA objective $\langle \psi_G(\theta) \rvert H_C \lvert \psi_G(\theta)\rangle$
$C_G^\star$	optimal MaxCut value of G
$r_G(\theta)$	approximation ratio $f_G(\theta)/C_G^\star$
Q	objective-query budget (number of distinct θ evaluated)
S	shots per query in the finite-shot model
$q_\phi(\theta \mid G)$	predicted Gaussian $\mathcal{N}(\mu_\phi(G), \Sigma_\phi(G))$
$\mu_\phi(G)$, $\Sigma_\phi(G)$	predicted mean and diagonal covariance
$\mathcal{T}_{\phi,\rho}(G)$	trust region of radius ρ
$\Sigma_{\text{post}}, \mu_{\text{post}}$	fused posterior covariance and mean
Σ_s, μ_s	source $s \in \{\text{GIN}, \text{loc}, \text{glob}, \text{TQA}\}$
δ_j	coordinate refinement step size for dimension j
$f(G) \in \mathbb{R}^8$	graph feature vector, (10)
θ^{TQA}	trotterized quantum annealing schedule, (9)
θ_i^{ref}	offline reference parameters for training graph i
$\alpha_G(\varepsilon)$	near-optimal proposal mass in theorem 1
L_G	local Lipschitz constant of f_G
λ	trust-region ridge term for numerical stability
η_{wd}	weight-decay coefficient

Supplementary Information

Uncertainty-Calibrated Trust Regions for Query-Efficient QAOA Parameter Search. This Supplementary Information collects the full notation, extended derivations and proofs, complete hyperparameter and training specifications, the high-performance-computing microbenchmark methodology and results, the Python/C++ cross-validation evidence, the reproducibility manifest, an extended discussion of related work, and a glossary. Every number reported here is produced by the deterministic artifact described in section 7.8; no quantity is hand-entered.

S1 Notation and symbols

Table S1 collects the symbols used throughout the manuscript. Vectors are column vectors; $\operatorname{diag}(\cdot)$ forms a diagonal matrix from a vector; $\|\cdot\|_2$ is the Euclidean norm; Π_{Θ_p} denotes modular projection onto the parameter domain Θ_p.

S2 Extended derivations and proofs

S2.1 Precision-weighted posterior fusion

We derive the four-source fusion rule used in algorithm 1. Treat each source s as supplying an independent Gaussian pseudo-likelihood $\mathcal{N}(\theta; \mu_s, \Sigma_s)$ over the angle vector θ. The product of Gaussians is, up to normalization, proportional to

$$\prod_s \exp\Big(-\tfrac{1}{2}(\theta-\mu_s)^\mathsf{T}\Sigma_s^{-1}(\theta-\mu_s)\Big) = \exp\Big(-\tfrac{1}{2}\big(\theta^\mathsf{T}\Sigma_{\text{post}}^{-1}\theta - 2\theta^\mathsf{T}\textstyle\sum_s \Sigma_s^{-1}\mu_s\big) + \text{const}\Big), \quad \text{(S1)}$$

where collecting the quadratic and linear terms gives the precision-weighted combination

$$\Sigma_{\mathrm{post}}^{-1} = \sum_s \Sigma_s^{-1}, \qquad \mu_{\mathrm{post}} = \Sigma_{\mathrm{post}} \sum_s \Sigma_s^{-1} \mu_s. \tag{S2}$$

Because all covariances are diagonal, (S2) factorizes over the $2p$ coordinates: for coordinate k, $[\mu_{\mathrm{post}}]_k = \big(\sum_s [\Sigma_s]_{kk}^{-1} [\mu_s]_k\big) / \big(\sum_s [\Sigma_s]_{kk}^{-1}\big)$. This shows explicitly that a source with small predicted variance on coordinate k dominates the fused mean on that coordinate, which is the intended behaviour: confident sources pull harder. The calibration floor $\Sigma'_{\mathrm{GIN}} = \max(\Sigma_{\mathrm{GIN}}, \Sigma_{\mathrm{loc}})$ prevents an overconfident neural head from collapsing the posterior onto an unreliable mean before any objective query is spent.

S2.2 Lipschitz lower bound on near-optimal mass

We justify the scaling relation $\alpha_G(\varepsilon) \gtrsim (\varepsilon/L_G)^d / (\rho^d \det(\Sigma_\phi + \lambda I)^{1/2})$ of section 6. Suppose f_G is locally L_G-Lipschitz near the trust-region optimum $\theta^{\mathcal{T}}$, so $f_G(\theta) \geq f_G^{\mathcal{T}} - L_G \|\theta - \theta^{\mathcal{T}}\|_2$. Then any θ within Euclidean distance ε/L_G of $\theta^{\mathcal{T}}$ is ε-optimal, i.e. the ball $B(\theta^{\mathcal{T}}, \varepsilon/L_G)$ is contained in $\mathcal{A}_{G,\varepsilon}$. For a proposal that is approximately uniform over the ellipsoidal trust region of volume $\propto \rho^d \det(\Sigma_\phi + \lambda I)^{1/2}$ (with $d = 2p$), the mass it assigns to $\mathcal{A}_{G,\varepsilon}$ is at least the volume ratio

$$\alpha_G(\varepsilon) \gtrsim \frac{\operatorname{vol} B(\theta^{\mathcal{T}}, \varepsilon/L_G)}{\operatorname{vol} \mathcal{T}_{\phi,\rho}(G)} \propto \frac{(\varepsilon/L_G)^d}{\rho^d \det(\Sigma_\phi + \lambda I)^{1/2}}, \tag{S3}$$

valid when ε/L_G is small relative to the trust-region extent so that the ball lies inside the region. Substituting into theorem 1 yields the query-count scaling: tighter calibrated covariance (smaller $\det(\Sigma_\phi + \lambda I)$) and smaller radius ρ both reduce the number of candidates needed to reach the ε-optimal set. The empirical $L_G \approx 114$ and volume ratio 4.72×10^{-4} of table 3 instantiate these constants on the benchmark.

S2.3 Full finite-shot argument

We expand the proof sketch of theorem 2. Let $\widehat{f}_{G,S}(\theta) = \frac{1}{S}\sum_{a=1}^{S} C_G(z_a)$ be the S-shot estimator with $z_a \sim |\langle z_a | \psi_G(\theta)\rangle|^2$ and $C_G(z_a) \in [0, |E|]$. By Hoeffding's inequality, for any fixed θ,

$$\Pr\big[\, |\widehat{f}_{G,S}(\theta) - f_G(\theta)| \geq t \,\big] \leq 2 \exp\Big(- \tfrac{2St^2}{|E|^2} \Big), \tag{S4}$$

so $\widehat{f}_{G,S}(\theta) - f_G(\theta)$ is sub-Gaussian with proxy variance σ_G^2/S and $\sigma_G^2 \leq |E|^2/4$ (assumption A3). Setting $t = \varepsilon/2$ and taking a union bound over the K evaluated candidates, the probability that any estimate deviates by more than $\varepsilon/2$ is at most $2K \exp(-S\varepsilon^2/(2|E|^2))$, which is $\leq \delta/2$ whenever $S \geq 2|E|^2 \log(4K/\delta)/\varepsilon^2$; using $\sigma_G^2 \leq |E|^2/4$ this is implied by the stated $S \geq 32\sigma_G^2 \log(4K/\delta)/\varepsilon^2$. By theorem 1 applied at tolerance $\varepsilon/2$ with $K \geq \log(2/\delta)/\alpha_G(\varepsilon/2)$, the best *exact* candidate is within $\varepsilon/2$ of $f_G^{\mathcal{T}}$ with probability $\geq 1 - \delta/2$. On the complement of both failure events (total probability $\geq 1 - \delta$), the candidate selected by the noisy objective is within $\varepsilon/2 + \varepsilon/2 = \varepsilon$ of $f_G^{\mathcal{T}}$, establishing the claim. This separates the parameter-query cost K from the shot cost S: a method can place good candidates with few queries yet still need many shots to rank them reliably, exactly the regime quantified by table 7.

Table S2: Complete hyperparameter specification.

Group	Hyperparameter	Value
GIN	message-passing layers L	4
	hidden width h	64
	MLP depth per layer	2 (layer-normalized)
	learnable $\varepsilon^{(\ell)}$	initialized to 0
	graph pooling	mean $\oplus$ max concatenation
	covariance floor $\sigma^2_{\min}$	10^{-4}
Training	optimizer	Adam [85]
	learning rate	3×10^{-4}
	schedule	cosine annealing
	max epochs	1500
	early-stopping patience	300
	gradient clip (ℓ_2)	1.0
	weight decay η_{wd}	10^{-4}
	loss	Gaussian NLL
Search	depth p (main)	3
	matched query budget Q	18
	local prior neighbours k	5
	step-size map	$\mathrm{clip}(0.5\sqrt{[\Sigma_{\mathrm{post}}]_{jj}}, 0.05, 0.30)$
	trust-region ridge λ	10^{-3}
	TQA total time T	validation-selected
Data	training graphs	240 (60/family)
	families	ER, 3-regular, BA, WS
	sizes n	$\{8, 10, 12, 14\}$
	test instances (main)	8 (2/family)
	global seed	`260424803`

S3 Complete hyperparameters and training protocol

Table S2 lists every hyperparameter required to reproduce the GIN predictor and the search policy. All values are fixed before test-time evaluation and selected on the validation split only.

S4 High-performance backend: methodology and full results

Methodology. The C++20 reference backend implements the same exact-statevector evaluator as the Python reference, with a narrow kernel boundary (table 1). Microbenchmarks are compiled with `-O3 -std=c++20 -march=native -pthread` on the platform of table 2 and report wall-clock time averaged over repeated batched evaluations after a warm-up pass. Thread scaling is measured on a fixed batch of 144 queries at n=14; per-query latency scaling is measured by sweeping n at single-thread settings.

Single-query cost scales as $\mathcal{O}(2^n)$. Table S3 reports single-thread mean per-query time as a function of n. Each increment of two qubits multiplies the cost by roughly four, consistent with the 2^n amplitude-vector length and confirming the complexity analysis of section 4.9.

Thread scaling. Table S4 reports speedup from batched parallel evaluation of 144 queries at n=14. The backend reaches 4.30× at 8 threads; the sublinear scaling beyond

Table S3: Single-thread mean per-query evaluation time of the C++ reference backend as a function of graph size n at p=2, Q=18. Each +2 qubits multiplies cost by ≈ 4, matching $\mathcal{O}(2^n)$.

n	Mean per-query (ms)	Statevector (bytes)
8	0.0103	4,096
10	0.0446	16,384
12	0.1996	65,536
14	0.8371	262,144
16	3.6947	1,048,576
18	15.9691	4,194,304

Table S4: Thread scaling of batched evaluation (144 queries, n=14).

Threads	Total time (ms)	Speedup
1	126.15	1.00×
2	71.10	1.77×
4	38.82	3.25×
8	29.33	4.30×

4 threads reflects memory-bandwidth pressure from the statevector sweeps rather than scheduling overhead.

Per-kernel cost. Table S5 decomposes the per-layer cost at n=14. The mixer kernel (strided 2×2 rotations over all qubits) is 2.60× more expensive than the bandwidth-bound cost-phase kernel, identifying it as the primary target for future SIMD/accelerator optimization.

Per-query latency invariance under threading. A single query's latency is essentially constant ($\approx$ 25.8 ms at n=14) across 1, 2, 4, and 8 threads, confirming that parallelism is harvested *across* the candidate batch rather than within a single statevector evaluation. This is the intended design: the optimizer emits a deterministic candidate stream and the backend parallelizes over independent graph–angle pairs without altering per-query numerics.

Per-family Python backend timing. Table S6 reports the Python reference-backend timing broken down by graph family at n=14, Q=18. Per-query time is stable across families ($\approx$ 1.40–1.50 ms), confirming that evaluation cost depends on the amplitude-vector length (2^n) and depth, not on graph topology; the small residual differences track edge count in the cost-phase kernel.

S5 Python/C++ cross-validation

A faster backend is scientifically admissible only if it reproduces the reference numerics bit-for-bit under the reproducibility contract. Table S7 reports the maximum absolute objective discrepancy between the Python and C++ evaluators and the thread-invariance error (the discrepancy between 1-thread and multi-thread C++ runs) across all tested (n, p) configurations. All discrepancies are at the level of double-precision round-off ($\sim 10^{-13}$) and the thread-invariance error is exactly zero, confirming deterministic reductions.

Table S5: Per-layer kernel cost at n=14.

Kernel	ms / layer	Characterization
Cost phase	0.1821	element-wise $e^{-i\gamma C(z)}\psi$, bandwidth-bound
Mixer	0.4736	strided 2×2 rotations over 14 qubits

Table S6: Python reference-backend timing by graph family at n=14, Q=18 (single thread). Per-query time is topology-independent; per-instance time includes feature extraction, candidate generation, and all 18 evaluations.

Family	Mean per-query (ms)	Per-instance (ms)
Erdős–Rényi	1.499	28.69
3-regular	1.401	26.71
Barabási–Albert	1.426	27.13
Watts–Strogatz	1.429	26.94

S6 Reproducibility manifest

Table S8 maps each reported quantity to the script that generates it. Running `python code/generate_all.py` regenerates all outputs end-to-end; `bash code/reproduce.sh smoke` runs a fast subset for continuous-integration checks. Every script is seeded by the global seed `260424803` and writes its outputs to `code/tables/` or `code/figures/`.

S7 Extended discussion of related work

This appendix expands section 3 along the axes most relevant to a machine-learning audience.

Amortized versus per-instance optimization. Bayesian optimization and evolutionary strategies are *per-instance*: they build a surrogate or population from scratch for every new graph and spend the budget refining it. UQ-QAOA is *amortized*: the cost of learning the trust-region geometry is paid once during offline training and reused across all test instances, so the entire test-time budget is spent on direct objective evaluation. The two are complementary—the learned trust region can seed a subsequent per-instance optimizer when additional budget is available.

Uncertainty as geometry versus uncertainty as a score. Most uncertainty-quantification work in deep learning treats predictive variance as a confidence score for downstream filtering or active learning. The contribution here is to treat the predicted diagonal covariance as an *operational metric*: it sets anisotropic step sizes, defines the admissible search ellipsoid, and (in the adaptive-budget mode) allocates the number of queries. This converts a distributional object into concrete search-policy decisions.

Position relative to learning-assisted QAOA. Prior learning-assisted QAOA methods predict a point or warm start and then hand control to a generic optimizer. The distinguishing feature here is the explicit geometric pathway from learned uncertainty to query allocation, combined with a physics-informed safety prefix that guarantees the policy never underperforms the strongest single-source prior at early budgets by construction.

Table S7: Python/C++ cross-validation: maximum absolute objective error and thread-invariance error. All configurations pass at double-precision tolerance.

n	p	Max abs. error	Thread-invariance error	Pass
8	3	4.33×10^{-13}	0	✓
8	6	3.47×10^{-13}	0	✓
8	7	2.52×10^{-13}	0	✓
10	3	4.80×10^{-14}	0	✓
10	6	7.82×10^{-14}	0	✓
10	7	3.57×10^{-13}	0	✓
12	3	4.76×10^{-13}	0	✓
12	6	2.17×10^{-13}	0	✓
12	7	2.81×10^{-13}	0	✓
14	3	5.29×10^{-13}	0	✓
14	6	3.13×10^{-13}	0	✓
14	7	3.55×10^{-14}	0	✓

Table S8: Artifact manifest: manuscript element → generating script.

Manuscript element	Generating script
table 4 (main benchmark)	`trace_evaluations.py`
table 5 (ablation)	`table03_ablation.py`
table 3 (bound verification)	`table02_bound_verification.py`
table 6 (p=6, 7)	`scripts / results/summary_p*.csv`
table 7 (finite shots)	finite-shot driver
table S3, S4, S5	`bench_cpu.py`, `bench_cpu.cpp`
table S7 (Py/C++ check)	`results/python_cpp_validation.csv`
fig. 1	`fig01_statevector_grid.py`
fig. 2	`scripts/make_figures.py`
fig. 3	`fig03_heldout_quality.py`
fig. 4	`fig04_trust_region_concept.py`

S8 Glossary

QAOA Quantum Approximate Optimization Algorithm: a depth-p variational quantum circuit optimized by a classical outer loop.

MaxCut the combinatorial problem of partitioning vertices to maximize the number (or weight) of cut edges; the benchmark objective.

GIN Graph Isomorphism Network: a maximally expressive first-order message-passing graph neural network.

TQA Trotterized Quantum Annealing: a depth-only linear-ramp schedule used as a physics-informed prior and early-budget safety prefix.

Trust region
a calibrated ellipsoid in parameter space, defined by the predicted covariance, to which local search is restricted.

Precision-weighted fusion
inverse-variance combination of independent Gaussian sources into a single posterior mean and covariance.

Query budget Q
the number of distinct parameter vectors evaluated on the (simulated) quantum objective; the primary resource accounted for.

Approximation ratio
objective value divided by the optimal cut value $C_G^\star$; the primary quality metric.

ECE Expected Calibration Error: a scalar summary of probabilistic miscalibration.

NLL Negative Log-Likelihood: the Gaussian training loss for the predictor.